

Advancing spatio-temporal processing through adaptation in spiking neural networks

Received: 2 August 2024

Accepted: 5 June 2025

Published online: 01 July 2025

 Check for updatesMaximilian Baronig ^{1,2,4}, Romain Ferrand^{1,2,4}, Silvester Sabathiel³ & Robert Legenstein ¹✉

Implementations of spiking neural networks on neuromorphic hardware promise orders of magnitude less power consumption than their non-spiking counterparts. The standard neuron model for spike-based computation on such systems has long been the leaky integrate-and-fire neuron. A computationally light augmentation of this neuron model with an adaptation mechanism has recently been shown to exhibit superior performance on spatio-temporal processing tasks. The root of the superiority of these so-called adaptive leaky integrate-and-fire neurons however is not well understood. In this article, we thoroughly analyze the dynamical, computational, and learning properties of adaptive leaky integrate-and-fire neurons and networks thereof. Our investigation reveals significant challenges related to stability and parameterization when employing the conventional Euler-Forward discretization for this class of models. We report a rigorous theoretical and empirical demonstration that these challenges can be effectively addressed by adopting an alternative discretization approach – the Symplectic Euler method, allowing to improve over state-of-the-art performances on common event-based benchmark datasets. Our further analysis of the computational properties of these networks shows that they are particularly well suited to exploit the spatio-temporal structure of input sequences without any normalization techniques.

Spiking neural networks (SNNs) have emerged as a viable biologically inspired alternative to artificial neural network (ANN) models¹. In contrast to ANNs, where neurons communicate analog numbers, neurons in SNNs communicate via digital pulses, so-called spikes. This event-based communication resembles the communication of neurons in the brain and enables highly energy efficient implementations in neuromorphic hardware^{2–6}. Recent advances in SNN research have shown that SNNs can be trained in a similar manner as ANNs using backpropagation through time (BPTT), leading to highly accurate models^{7,8}.

The dominant spiking neuron model used in SNNs is the leaky integrate and fire (LIF) neuron⁹. The LIF neuron has a single state variable $u(t)$ that represents the membrane potential of a biological neuron. Incoming synaptic currents are integrated over time in a leaky manner on the time scale of tens of milliseconds. Once the membrane potential reaches a threshold ϑ , the membrane potential is reset and the neuron emits a spike (i.e., its output is set to 1). The leaky integration property of the LIF neuron model reproduces the sub-threshold behavior of so-called excitability class 1 neurons in the brain (Fig. 1a, left)^{10,11}.

¹Institute of Machine Learning and Neural Computation, Graz University of Technology, Graz, Austria. ²TU Graz—SAL Dependable Embedded Systems Lab, Silicon Austria Labs, Graz, Austria. ³Silicon Austria Labs GmbH, Graz, Austria. ⁴These authors contributed equally: Maximilian Baronig, Romain Ferrand.

✉ e-mail: robert.legenstein@tugraz.at

A second class of neurons called excitability class 2 neurons (Fig. 1a, right) exhibit more complex dynamics with sub-threshold membrane potential oscillations (Fig. 1b) and spike frequency adaptation (SFA, Fig. 1c). Such complex dynamics cannot be modeled with the single state variable $u(t)$ of the LIF neuron. Pioneering work has shown that these behaviors can be reproduced by a simple extension of the LIF neuron model that adds a second state variable to the neuron dynamics which interacts with the membrane potential in a—typically negative—feedback loop¹². Neuron models of this type are called adaptive LIF neurons.

With the growing interest in SNNs for neuromorphic systems, researchers have started to train recurrent SNNs (RSNNs) consisting of adaptive LIF neurons with BPTT on spatio-temporal processing tasks. First results were based on neuron models that implement a threshold adaptation mechanism, where the second state variable is a dynamic threshold $\vartheta(t)$ ^{13–15}. Each spike of the neuron leads to an increase of this threshold, which implements the negative feedback loop mentioned above and leads to SFA (Fig. 1c). The performance of these models clearly surpassed those achieved with networks of LIF neurons while being highly efficient on neuromorphic hardware with orders of magnitudes energy savings when compared to implementations on CPUs or graphical processing units (GPUs)⁶.

While threshold adaptation implements SFA, the resulting neuron model still performs a leaky integration of input currents and does not exhibit the typical sub-threshold membrane potential oscillations of class 2 neurons. Hence, more recent work considered networks of neurons with a form of adaptation often referred to as sub-threshold or current-based adaptation. The second state variable $w(t)$ is interpreted as negative adaptation current that is increased not only by neuron spikes but also by the sub-threshold membrane potential itself. This sub-threshold feedback leads to complex oscillatory membrane potential dynamics (Fig. 1b). Interestingly, simulation studies have shown that SNNs equipped with sub-threshold adaptation achieve significantly better performances than SNNs with threshold adaptation^{15–17}.

Although the converging evidence suggests that networks of adaptive LIF neurons are superior to LIF networks for neuromorphic applications, there are still many questions open. First, to achieve top performance, usually all neuron parameters are trained together with the synaptic weights. Changes of the neuron parameters however can quickly lead to unstable models which disrupts training. To avoid instabilities, parameter bounds have to be defined and fine-tuned. If these bounds are too wide, the network can become unstable, if they are too narrow, one cannot utilize the full computational expressivity.

In this work, we show that this problem is not inherent to the neuron model but rather caused by the standard discrete-time formulation of the continuous neuron dynamics which is based on the Euler-Forward discretization method. Despite mere stability issues, we identified a plethora of drawbacks arising from the application of the widely used Euler-Forward discretization. These include, inter alia, unintended interdependencies between neuron parameters, deviations of the discrete model dynamics from its continuous counterpart, limitations in neuron expressibility, strong dependence between the discretization time step and the neuron dynamics, non-trivial divergence boundaries. Our thorough theoretical analysis reveals that the alternative, even simple Symplectic-Euler discretization method remarkably alleviates the drawbacks of the Euler-Forward method almost entirely, without additional computational cost or implementation complexity. While this discretization method could in principle be applied to an entire family of multi-dimensional neuron models, we mainly focus our theoretical and empirical analyses on a specific adaptive neuron model that recently gained traction. Using this insight, we demonstrate the power of adaptation by showing that our improved adaptive RSNNs outperform the state-of-the-art on spiking speech recognition datasets as well as an ECG dataset. We then

show that the superiority of adaptive RSNNs is not limited to classification tasks but extends to the prediction and generation of complex time series. Second, there is a lack of understanding why sub-threshold adaptation is so powerful in RSNNs. We thoroughly analyze the computational dynamics in single adaptive LIF neurons, as well as in networks of such neurons. Our analysis suggests that adaptive LIF neurons are especially capable of detecting temporal changes in input spike density, while being robust to shifts of total spike rate. Hence, adaptive RSNNs are well suited to analyze the temporal properties of input sequences. Third, high-performance SNNs are usually trained using normalization techniques such as batch normalization or batch normalization through time^{18–21}. These methods however complicate the training process and the implementation of networks on neuromorphic hardware. We show that adaptation has a previously unrecognized benefit on network optimization. Since adaptation inherently stabilizes network activity, we hypothesized that explicit normalization is not necessary in adaptive RSNNs. In fact, all our results were obtained without explicit normalization techniques. We test this hypothesis and show that in contrast to LIF networks, networks of adaptive LIF neurons can tolerate substantial shifts in the mean input strength as well as substantial levels of background noise even when these perturbations were not observed during training.

Results

Adaptive LIF neurons

The leaky integrate-and-fire (LIF) neuron model⁹ evolved as the gold standard for spiking neural networks due to its simplicity and suitability for low-power neuromorphic implementation²². The continuous-time equation for the membrane potential of the LIF neuron at time t is given by

$$\tau_u \dot{u} = -u(t) + I(t), \quad (1)$$

where τ_u is the membrane time constant, and $I(t) = \sum_i \theta_i x_i(t)$ the input current composed of the sum of neuron inputs $x_i(t)$ scaled by the corresponding synaptic weights θ_i . A dot above a variable denotes its derivative with respect to time. If the membrane potential $u(t)$ crosses the spike threshold ϑ from below, a spike is emitted and u is reset to the reset potential. In the absence of input, $u(t)$ decays exponentially to zero. The LIF neuron equation models so-called integrating class 1 neurons in the brain (Fig. 1a), which are integrating incoming currents in a leaky manner. The simple first-order dynamics however does not allow the LIF model to account for another class of neurons frequently occurring in the brain: resonating/oscillating class 2 neurons (Fig. 1a, b). In contrast to integrators, such neurons exhibit oscillatory behavior in response to stimulation, giving rise to interesting properties entirely neglected by LIF neurons. Such oscillatory behavior is often modeled by adding a second time-varying variable — the adaptation current $w(t)$ — to the neuron state^{12,16,23,24}. The resulting neuron model, which we refer to as adaptive leaky integrate-and-fire (adLIF) neuron, has significant advantages over LIF neurons in terms of feature detection capabilities and gradient propagation properties, as we show in the next few sections. The adLIF model is described in terms of two coupled differential equations

$$\tau_u \dot{u} = -u(t) + I(t) - w(t) \quad (2)$$

$$\tau_w \dot{w} = -w(t) + au(t) + bz(t), \quad (3)$$

where τ_w is the adaptation time constant and a and b are adaptation parameters, defining the behavior of the neuron. When comparing the LIF equation (1) with equation (2), we see that the latter resembles the LIF dynamics where the adaptation current $w(t)$ is subtracted, with its dynamics defined in equation (3).

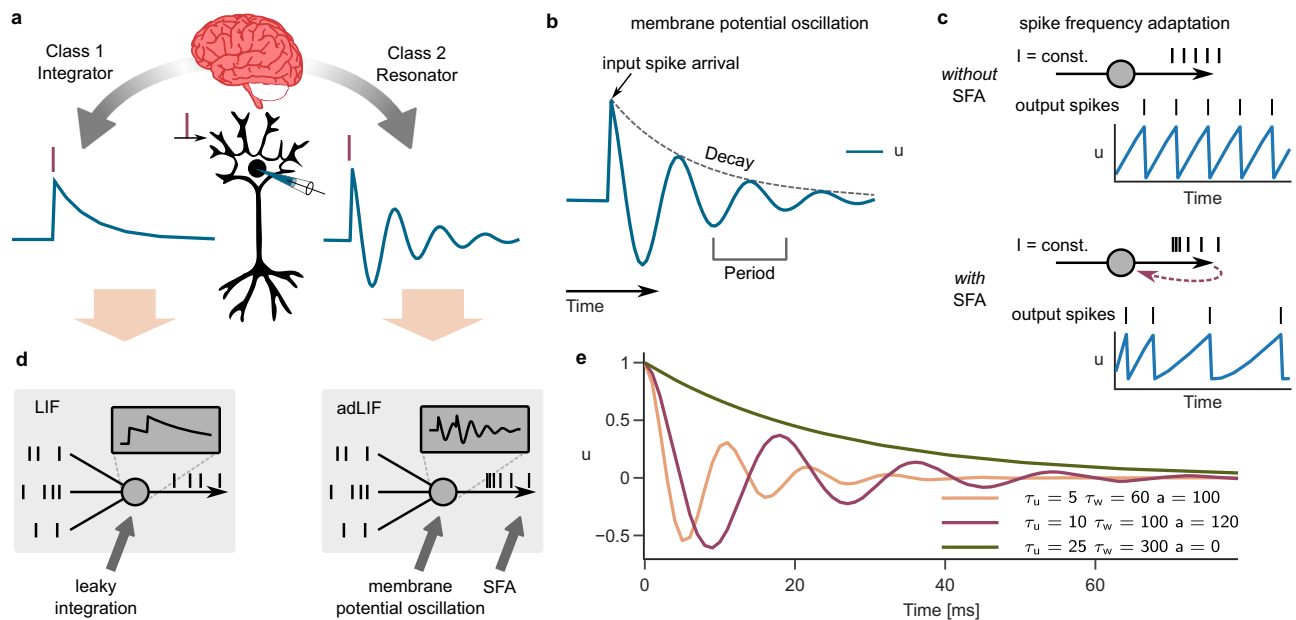


Fig. 1 | The adLIF neuron with membrane potential oscillation and spike-frequency adaptation. **a** Neurons in the brain have been classified into two excitability classes, integrators (class 1) and resonators (class 2). While resonators show membrane potential oscillations, integrators do not. **b** Membrane potential oscillation in response to an input pulse. The period P and decay r are sufficient to fully characterize the oscillating spike response. **c** Example of a neuron without

(top) and with (bottom) spike frequency adaptation (SFA). Dotted arrow illustrates the feed-back of output spikes to the neuron membrane state in the adLIF model. **d** Adaptive LIF (adLIF) neurons (see Eq. (2)) differ in 2 features from vanilla LIF neurons: membrane potential oscillations and SFA. **e** Impulse response functions for different parameterizations of adLIF neurons. For $a = 0$ (green), no oscillations occur and the spike response reduces to leaky integration.

The parameter $a \in \mathbb{R}$ scales the coupling of the membrane potential $u(t)$ with the adaptation current $w(t)$. The negative feedback loop between $u(t)$ and $w(t)$ defined by Equations (2) and (3) leads to oscillations of the membrane potential for large enough a , see Fig. 1b. The oscillation can be characterized by the decay rate r and the period $P = \frac{1}{f}$ given by the inverse of the intrinsic frequency f . As we discuss later in the manuscript, f characterizes the frequency tuning of the neuron, whereas the decay rate r is an indicator of its stability and time scale.

The parameter $b \geq 0$ weights the feed-back from the neuron's output spike $z(t)$ onto the adaptation variable $w(t)$. Hence, each spike has an inhibitory effect on the membrane potential, which leads to spike frequency adaptation (SFA)¹⁵, see Fig. 1c. We refer to this auto-feedback governed by parameter b as spike-triggered adaptation in the following. SFA has also been implemented directly using an adaptive firing threshold that is increased with every output spike^{13,25}. In contrast to the adLIF model, these models do not exhibit membrane potential oscillations.

The adLIF model combines both membrane potential oscillations and SFA in one single neuron model (Fig. 1d). Depending on the parameters, adLIF neurons can exhibit oscillations of diverse frequencies and decay rates (Fig. 1e), and are equivalent to LIF neurons for $a, b = 0$, where neither oscillations nor spike-triggered adaptation occur. A reduced variant of the adLIF neuron is given by the resonate-and-fire neuron¹².

Originally developed to efficiently replicate firing patterns of biological neurons, the adLIF model recently gained attention due to significant performance gains over vanilla LIF neurons in several benchmark tasks, despite its little computational overhead^{15,16,24}. In particular, gradient-based training of networks of adLIF neurons on spatio-temporal processing tasks appears to synergize well with oscillatory dynamics. However, these empirical findings are so far not accompanied by a good understanding of the reasons for this superiority.

When comparing the responses of the LIF and adLIF neuron, an important computational consequence of membrane potential oscillations has been noted: In contrast to the LIF neuron, which responds

with higher amplitude of u to higher input spike frequency (Fig. 2a), the adLIF neuron is most strongly excited if the frequency of input spikes matches the intrinsic frequency f of the neuron (Fig. 2b, c, see also^{12,17,23}). To demonstrate this resonance phenomenon, we show the membrane potential of an adLIF neuron with intrinsic frequency $f = 60$ Hz for an input spike triplet exactly at this intrinsic frequency f (Fig. 2b, left), compared to a spike triplet of higher rate (Fig. 2b, right). The resulting amplitude of the membrane potential u is higher in the former case, indicating resonance. Fig. 2c shows that the neuron exhibits a frequency selectivity specifically for its intrinsic frequency f .

We took this analysis a step further and asked whether this resonance could account for frequencies in the input spike train that are not directly encoded by spike rate, but rather by slow changes of the spike rate over time, a coding scheme previously termed spike frequency modulation (SFM)²⁶. As a guiding example, we encoded a slowly-varying sinusoidal signal as a spike train, where the magnitude of the signal at a certain time is given by the local spike rate, shown in Fig. 2d. The spike rate thereby varied between 0 Hz and 200 Hz, whereas the underlying, encoded sinus signal oscillated with a constant frequency of 10 Hz. Again, we see increased response of the membrane potential u over time in the case of the 10 Hz input compared to a slower 7 Hz sinus signal (Fig. 2d, e), due to resonance with the adLIF neuron, see also Fig. 2f. In contrast, the corresponding membrane voltage response amplitude of a LIF neuron is almost indifferent to the intrinsic frequency of the underlying sinus input, see Fig. 2g. This shows that in contrast to the LIF neuron, the adLIF neuron model is sensitive to the longer-term temporal structure, i.e. variation of the input signal. In Section Computational properties of adLIF networks, we highlight the importance of this frequency-dependence of neuron responses as a key ingredient for the powerful feature detection capabilities of networks of adLIF neurons.

The Symplectic-Euler discretized adLIF neuron

In the previous section, we defined the LIF and adLIF neuron models via continuous-time, ordinary differential equations. In practice, it is

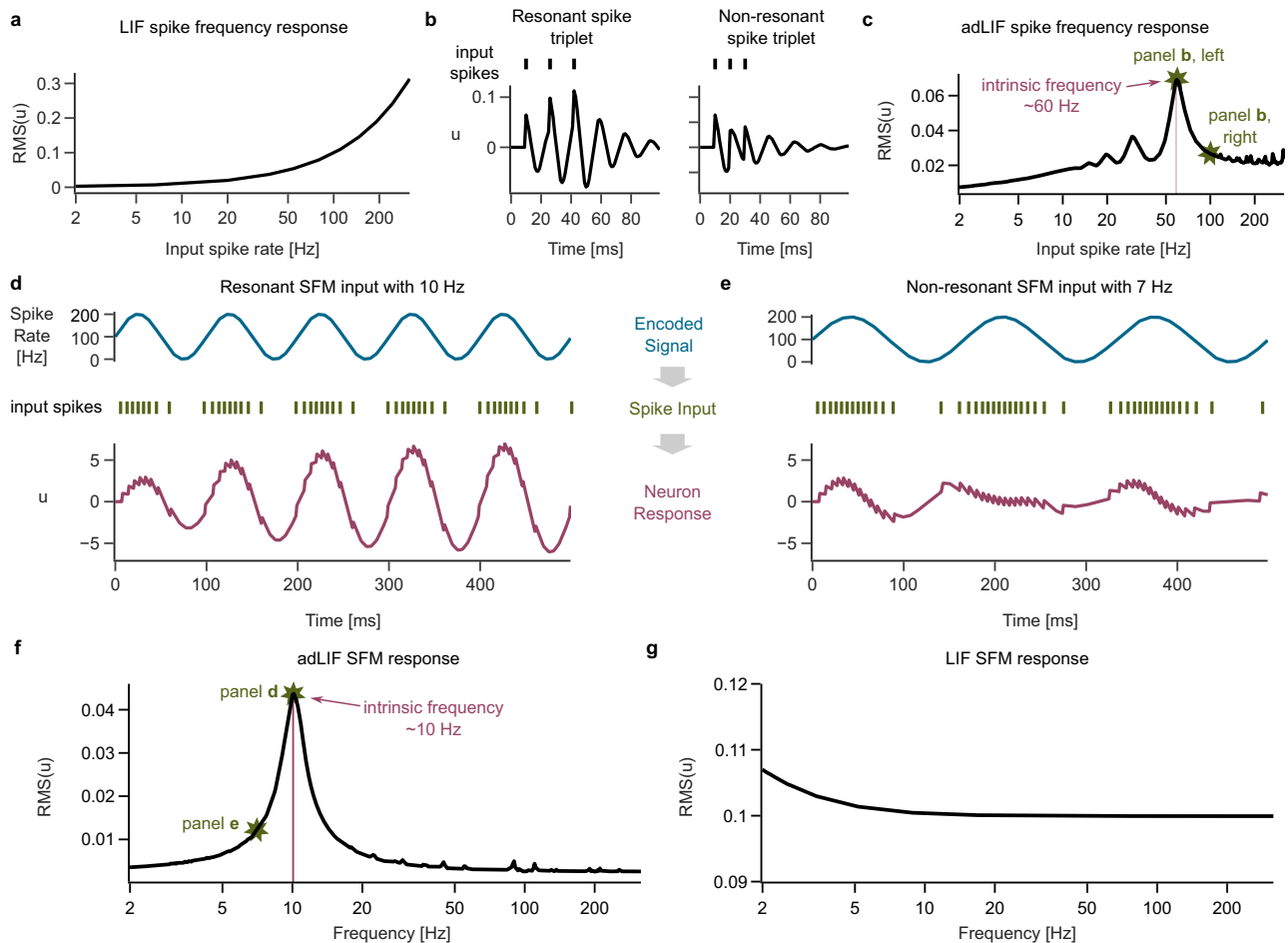


Fig. 2 | The adLIF neuron model shows frequency-selective resonating behavior. **a** Voltage response (root mean squared membrane potential over 10 seconds) of a LIF neuron in response to tonic spike input of different rates. **b** Membrane potential response of an adLIF neuron with intrinsic frequency of $f \approx 60$ Hz to an input spike triplet at 60 Hz (left) and 100 Hz (right). **c** Voltage response of the same adLIF neuron as in panels (b) and (c) for tonic spiking input at different rates. The stars indicate the frequencies shown in panels (b) and (c). **d** A 10 Hz sinusoidal input

signal (top) is encoded as an input spike train (middle) through spike frequency modulation (SFM), see main text for details. Membrane potential response of an adLIF neuron with intrinsic frequency $f \approx 10$ Hz (bottom). **e** Same as panel (d), but for a sinusoidal input at 7 Hz. **f** Voltage response of an adLIF neuron to SFM-encoded sinusoidal input at various frequencies. The stars indicate the frequencies shown in panels (d) and (e). **g** Same as panel (f), but for a LIF neuron. See “Methods” for parameters and input generation.

however standard to discretize the continuous-time dynamics of the spiking neuron model. This allows not only to use powerful auto-differentiation capabilities of machine learning software packages such as TensorFlow²⁷ or PyTorch²⁸, but also for implementation of such neuron models in discrete-time operating neuromorphic hardware²⁹. Discretization of the LIF neuron model Eq. (1) is straight-forward. In contrast, for the adLIF model, the interdependency of the two state variables during a discrete time step Δt cannot be taken into account exactly in a simple manner (the exact solution involves a matrix exponential). Nevertheless, for efficient simulation and hardware implementations, simple update equations are needed. Therefore, approximate discrete update equations for the membrane potential u and the adaptation current w are usually obtained by the Euler-Forward method¹⁶. In the following we analyze discretization methods for adLIF neurons through the lens of dynamical systems analysis. We find that the Euler-Forward method is problematic, and propose the utilization of a more stable alternative discretization method.

A common approach to study dynamical systems is through the state-space representation, which recently gained popularity in the field of deep learning^{30,31}. Re-formulation of spiking neuron models in a canonical state-space representation provides a convenient unified way to study their dynamical properties. The continuous-time equations of the adLIF neuron Eq. (2), (3) can be re-written in such a state-

space representation as a 2-dimensional linear time-invariant (LTI) system with state vector $\mathbf{s}$ as

$$\begin{aligned} \dot{\mathbf{s}}(t) &= \begin{pmatrix} \dot{u}(t) \\ \dot{w}(t) \end{pmatrix} = \mathbf{A}\mathbf{s}(t) + \mathbf{B}\mathbf{x}(t) \\ &= \underbrace{\begin{pmatrix} -\frac{1}{\tau_u} & -\frac{1}{\tau_u} \\ \frac{a}{\tau_w} & -\frac{1}{\tau_w} \end{pmatrix}}_{\mathbf{A}} \begin{pmatrix} u(t) \\ w(t) \end{pmatrix} + \underbrace{\begin{pmatrix} \frac{1}{\tau_u} & 0 \\ 0 & \frac{b}{\tau_w} \end{pmatrix}}_{\mathbf{B}} \begin{pmatrix} I(t) \\ z(t) \end{pmatrix}, \end{aligned} \quad (4)$$

with system matrix $\mathbf{A}$ and input matrix $\mathbf{B}$. This equation only describes the sub-threshold dynamics of the neuron (i.e., it holds as long as the threshold is not reached). The reset can be accounted for by the threshold condition: When the voltage crosses the firing threshold ϑ , an output spike is elicited and the neuron is reset. The goal of discretization is to obtain discrete-time update equations of the form

$$\mathbf{s}[k] = \begin{pmatrix} u[k] \\ w[k] \end{pmatrix} = \bar{\mathbf{A}}\mathbf{s}[k-1] + \bar{\mathbf{B}}\mathbf{x}[k], \quad (5)$$

where $f[k]$ denotes the value of state variable f at discrete time step k , i.e., $f[k] \equiv f(\Delta t k)$ for discrete time increment Δt and integer-valued $k > 0$. Here, $\bar{\mathbf{A}}$ and $\bar{\mathbf{B}}$ denote the state and input matrix of the discrete

time system respectively. In the SNN literature, the most commonly used approach to obtain the discrete approximation to the continuous system from Eq. (4) is the Euler-Forward method^{16,17,24}, which results in update equations

$$\hat{u}[k] = \alpha u[k-1] + (1-\alpha)(-w[k-1] + I[k]) \quad (6a)$$

$$w[k] = \beta w[k-1] + (1-\beta)(\alpha u[k-1] + bS[k]), \quad (6b)$$

where $\hat{u}$ denotes the membrane potential before the reset is applied, $\alpha = 1 - \frac{\Delta t}{\tau_u}$, and $\beta = 1 - \frac{\Delta t}{\tau_w}$. The spike output of the neuron is given by

$$S[k] = \begin{cases} 1 & \text{if } \hat{u}[k] > u_{th} \\ 0 & \text{otherwise.} \end{cases} \quad (7)$$

Finally, $u[k]$ is obtained by applying the reset to $\hat{u}$ via

$$u[k] = \hat{u}[k] \cdot (1 - S[k]). \quad (8)$$

This Euler-Forward discretization yields the discrete state-space matrices

$$\bar{A}_{EF} = \begin{pmatrix} \alpha & -(1-\alpha) \\ \alpha(1-\beta) & \beta \end{pmatrix}, \bar{B}_{EF} = \begin{pmatrix} (1-\alpha) & 0 \\ 0 & b(1-\beta) \end{pmatrix} \quad (9)$$

for $\bar{A}$ and $\bar{B}$ in Eq. (5). In practice (see for example¹⁶), the coefficients α and β are often replaced by exponential decay terms $\alpha = \exp\left(-\frac{\Delta t}{\tau_u}\right)$ and $\beta = \exp\left(-\frac{\Delta t}{\tau_w}\right)$ akin to the LIF discretization, as it is exact in the latter case. However, for adLIF neurons, the Euler-Forward approximation is quite imprecise which can quickly result in unstable and diverging behavior of the system, as we will show below. A better approximation is given by the bilinear discretization method, a standard method also used in state space models³⁰, which is however computationally more demanding. An alternative is the Symplectic-Euler (SE) method³² that has previously been used in non-spiking oscillatory systems³³. We found that the Symplectic-Euler (SE) discretization provides major benefits in terms of stability, expressivity, and trainability of the adLIF neuron, while being computationally as efficient as Euler-Forward. The SE method has been shown to preserve the energy in Hamiltonian systems, a desirable property of a discretization of such systems³². As we show below, the improved stability of the SE method still applies to the adLIF neuron model, even though it is non-Hamiltonian. The SE method is similar to the Euler-Forward method, with the only difference that one computes the state variable $w[k]$ from $u[k]$ instead of $u[k-1]$, resulting in the discrete dynamics

$$\hat{u}[k] = \alpha u[k-1] + (1-\alpha)(-w[k-1] + I[k]) \quad (10a)$$

$$w[k] = \beta w[k-1] + (1-\beta)(\alpha u[k] + bS[k]). \quad (10b)$$

We refer to this neuron model as the SE-adLIF model in order to distinguish it from the Euler-Forward discretized model. Note that the reset mechanism from Eq. (8) is applied to obtain $u[k]$ from $\hat{u}[k]$ before computing $w[k]$. While it is also possible to apply the reset after computing $w[k]$, we found that the above described way yields the best performance.

For the sub-threshold dynamics, this leads to update matrices (see Section Derivation of matrices $\bar{A}_{SE}$ and $\bar{B}_{SE}$ for the SE-adLIF neuron in

Methods)

$$\bar{A}_{SE} = \begin{pmatrix} \alpha & -(1-\alpha) \\ \alpha(1-\beta)\alpha & \beta - \alpha(1-\beta)(1-\alpha) \end{pmatrix} \quad (11)$$

$$\bar{B}_{SE} = \begin{pmatrix} (1-\alpha) & 0 \\ 0 & b(1-\beta) \end{pmatrix}$$

for the discrete state-space formulation given by Eq. (5).

Stability analysis of discretized adLIF models

A desirable characteristic of a discretization method is its ability to maintain a close alignment between the discretized system and the continuous ground-truth. In contrast to SE, the EF discretization exhibits a pronounced dependence of this alignment on the discretization time step Δt , thereby reducing its robustness. This is visualized in Fig. 3a, where we discretized an adLIF neuron with the EF method (left) and the SE method (right) using 3 different discretization time steps Δt . We observed that the EF-discretized neuron clearly diverges for larger values of Δt , in this example even for $\Delta t = 1$ ms. In contrast, the same neuron discretized with the Symplectic Euler method is robust to the choice of Δt . The divergence of the neuron can be quantified by its decay rate r , which gives the exponential decay of the envelope of a neuron's membrane potential $u(t)$ (see also Fig. 1b). For $r < 1$, the neuron stably decays to a resting-state equilibrium. However, if this decay rate exceeds 1, the neuron becomes unstable and its membrane potential grows indefinitely, as observable for the EF-adLIF neuron with $\Delta t = 1$ in Fig. 3a. When comparing the relationship of the decay rate r with respect to discretization time step Δt , as visualized in Fig. 3b, the favorable adherence of the SE-discretized adLIF to the continuous model is evident. The SE-adLIF decay rate is independent of Δt and evaluates to $r \approx 0.972$, which is the decay rate of the continuous model. For the EF discretization in contrast, r grows along with increasing Δt , resulting in discretized neurons exceeding the stability boundary at $r = 1$. As the computational cost of training SNNs via BPTT increases with smaller discretization time steps due to longer sequence lengths, the SE discretization is clearly favorable over EF, since it ensures stability and adherence to the continuous ground truth when Δt is large. The adherence of SE-adLIF to the continuous adLIF model is not limited to its robustness to the choice of Δt . For a given $\Delta t = 1$ ms, SE-adLIF follows the characteristics of the continuous model with respect to its parameters τ_u , τ_w , and a much closer. While time constants τ_u and τ_w affect the decay rate r of the adLIF neuron, parameter a determines the frequency of oscillation. Since the continuous-time adLIF neuron model Eq. (2) is inherently stable for $a \geq -1$ (see Section Proof of stability bounds for the continuous adLIF model in Methods for proof), it is a desirable property of a discretization method to preserve this stability for all possible parameterizations.

This is shown empirically in Fig. 3c. We instantiated 1000 different neurons for both discretization methods, Euler-Forward and SE, as well as the continuous model, in a grid-like manner over a reasonable parameter range and plot their calculated frequencies and decay rates in Fig. 3c. While the continuous system is stable for all considered parameter combinations (middle panel), the Euler-Forward approximation (left panel) is unstable for many parameterizations (decay rate $r > 1$). In contrast, for the SE discretization (right panel), all parameter combinations resulted in stable neuron dynamics ($r < 1$). These empirical results show that the SE discretization more closely follows the stability properties of the continuous model, whereas the Euler-Forward method deviates drastically from both. How can this discrepancy be explained?

When analyzing the discretized neurons, one has to calculate f and r directly from the discrete system by calculating the eigenvalues $\lambda_{1,2}$ of the state transition matrix $\bar{A}$. This allows to study the behavior and the stability of the adLIF neuron model for different discretizations. Two cases have to be differentiated: If the eigenvalues are complex, the membrane potential exhibits oscillations, whereas if they are real, no

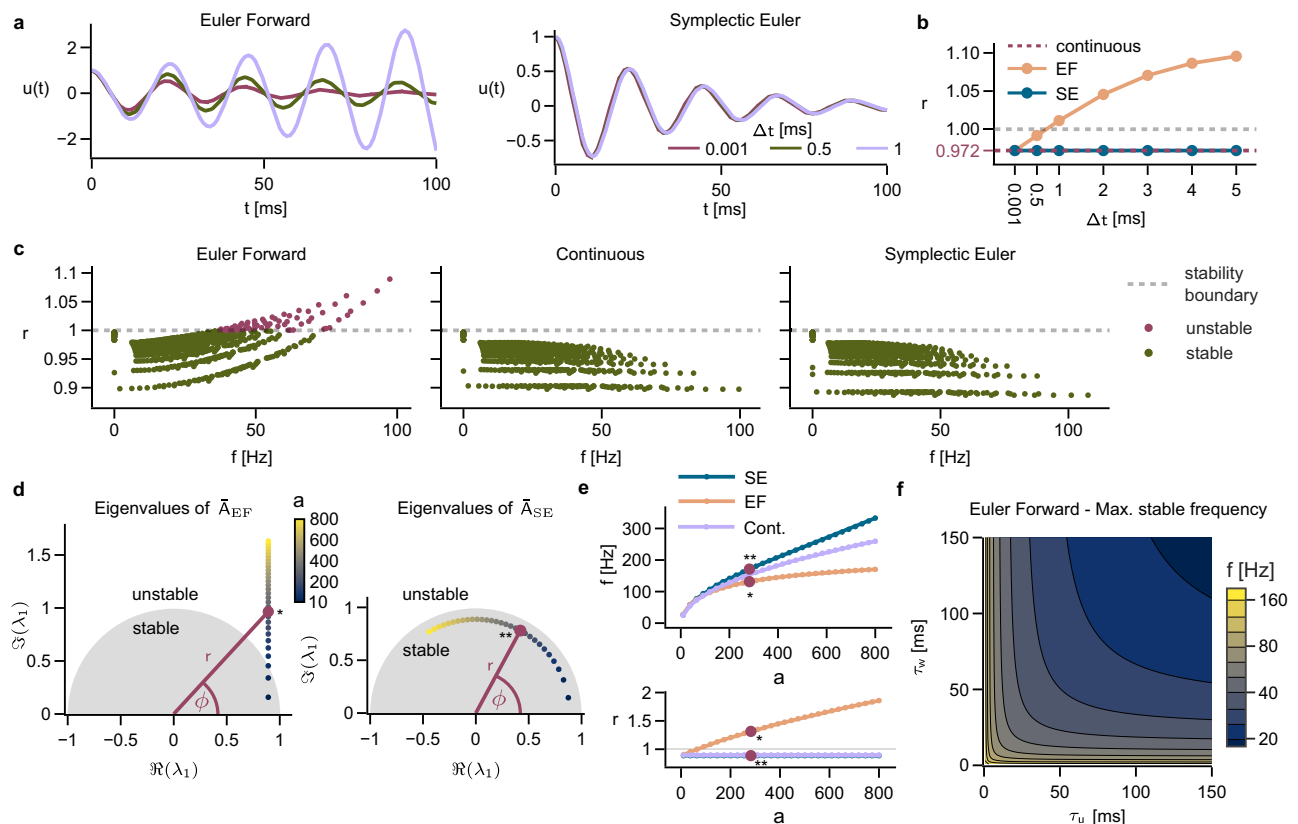


Fig. 3 | Stability of adLIF discretizations. **a** Membrane potential $u(t)$ over time for an EF-adLIF (left) and a SE-adLIF (right) neuron for different discretization time steps $\Delta t \in \{0.001, 0.5, 1\}$. Both neurons have the same parameters ($\tau_u = 25$ ms, $\tau_w = 60$ ms, $a = 120$). **b** Relationship between the decay rate r and discretization time step Δt for adLIF models with different discretizations, EF and SE. All decay rates are calculated with respect to 1 ms, a decay rate of $r = 0.9$ hence represents a decrease in magnitude of 10% every 1 ms. The decay rate ($r = 0.972$) of the equivalently parameterized continuous model is highlighted. Same neuron parameters as in panel (a). **c** Intrinsic frequency f and per-timestep decay rate r of 1000 different parameterizations of adLIF neurons for Euler-Forward discretization (left), SE (right), and the continuous model (middle). The horizontal dotted line at $r = 1.0$ marks the stability bound. Instances above this line diverge due to exponential growth. Parameter ranges are uniformly distributed over the intervals $a \in [0, 120]$, $\tau_u \in [5, 25]$ ms and $\tau_w \in [60, 300]$ ms. **d** Eigenvalues of $\bar{A}_{EF}$ (left) and $\bar{A}_{SE}$ (right)

plotted in the complex plane for fixed $\tau_u = 25$ ms, $\tau_w = 60$ ms and varying $a \in [10, 800]$. Decay rate r as modulus of the eigenvalue λ_1 and angle ϕ as argument of λ_1 are shown for $a = 282$, marked with * and ** for EF and SE respectively. The gray half-circle denotes the stable region of $r \leq 1$. **e** Relationship of parameter a to intrinsic frequency f (top) and decay rate r (bottom) for the same τ_u and τ_w as in panel (d). Points with * and ** denote the corresponding eigenvalues from panel (d). Recall the linear relationship $f = \frac{\phi}{2\pi\Delta t}$ between angle ϕ and f of the discrete models. Horizontal gray line in bottom panel denotes stability boundary of $r = 1$. Values for r of continuous model ($r = 0.997$) and SE-adLIF ($r = 0.887$) are constant w.r.t. a (SE-adLIF values for r not visible due to near-perfect fit to the continuous model). **f** Maximum admissible frequency for stable dynamics for Euler-Forward discretization with $\Delta t = 1$ ms over different values of τ_u and τ_w , where a is set to the maximum stable value $a_{\max}$ (see main text and Section Stable ranges for intrinsic frequencies of EF-adLIF in “Methods”).

oscillations occur and the neuron behaves similar to a LIF neuron. In the complex case, we can write the eigenvalues in polar form as $\lambda_{1,2} = re^{\pm j\phi}$, where j denotes the imaginary unit. Hence, the eigenvalues are complex conjugates, the decay rate r is given by their modulus (see Fig. 3d) and ϕ is obtained as the argument of λ_1 ($\arg(\lambda_2) = -\phi$). Intuitively, the angle ϕ is the rotation of the neuron state with each time step Δt in radians, and hence determines the frequency of the oscillation. One thus obtains the intrinsic frequency f in Hertz as $f = \frac{\phi}{2\pi\Delta t}$. In the case of real eigenvalues, r is given by the magnitude of the largest eigenvalue. AdLIF neurons can thereby represent underdamped (complex eigenvalues), critically damped (equal real eigenvalues), and overdamped (non-equal real eigenvalues) systems via different parameterizations. Note, that only in the underdamped case the neuron can oscillate.

As described above, the eigenvalues $\lambda_{1,2}$ of state transition matrices $\bar{A}_{EF}$ and $\bar{A}_{SE}$ are determining the stability of the discrete neurons. We can directly observe the origin of instability for the EF-adLIF by plotting eigenvalues for different neuron parameters in the complex plane. In Fig. 3d, we show some eigenvalues of $\bar{A}_{EF}$ and $\bar{A}_{SE}$ in the case of fixed time constants τ_u and τ_w and varying parameter a . In

the complex plane, the stability boundary appears as circle, separating the stable ($r < 1$, gray area) from the unstable region ($r > 1$). Our analysis in Section Derivation of stability bounds for EF-adLIF in Methods shows that for the Euler-Forward method, for fixed time constants τ_u and τ_w , the real part $\Re(\lambda_{1,2})$ of these eigenvalues is constant and strictly positive with respect to a , such that the eigenvalues are aligned along a vertical line in the right half-plane of the complex plane (see Fig. 3d, left panel). As a increases, the imaginary part increases and so does the decay rate r . As already mentioned, in the continuous adLIF model, parameter a only affects the frequency of oscillation, but not the decay rate. The SE-adLIF model adheres to this property, since the modulus of the eigenvalues does not change with respect to a . For EF-adLIF however, parameter a exhibits an undesired side-effect on the modulus. Hence, for the Euler-Forward discretized neuron, the eigenvalues overshoot the stability boundary $r = 1$ for increasing a . This leads to a drastically reduced range of the angle ϕ and therefore a reduced range of the intrinsic frequency f where the neuron is stable.

In contrast, for the SE discretized model, the parameter a controls only the angle ϕ of the eigenvalues (see Section Derivation of the stability bounds of SE-adLIF in Methods) and hence the intrinsic

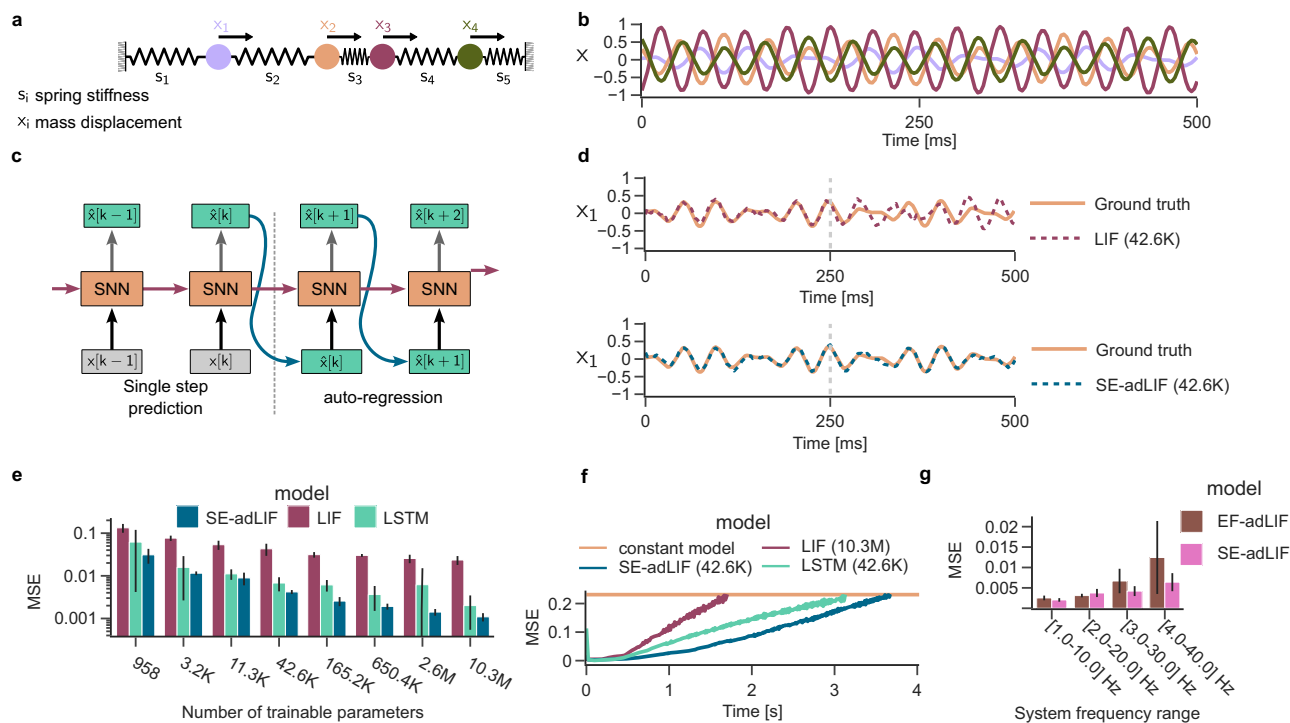


Fig. 4 | Prediction and generation of complex oscillatory dynamics. **a** Schematic of a 4-degree-of-freedom spring-mass system. x_1 to x_4 represent the displacements of the four masses. **b** Example displacement dynamics generated over a period of 500 ms. **c** Illustration of the auto-regression task. For the first 250 ms, the network received the true displacements $x[k]$ and predict the next displacement $\hat{x}[k+1]$. After 250 ms, the model generates the displacements by using its own predictions from the previous time step in an autoregressive manner. **d** Displacement predictions for mass x_1 , by a LIF (top) and adLIF (bottom) network with 42.6K trainable parameters. **e** Mean squared error (MSE) in logarithmic scale during the auto-

regression period for LIF, adLIF, and LSTM networks of various sizes (mean and STD over 5 unique randomly generated spring-mass systems). **f** Divergence of generated dynamics in the auto-regressive phase (starting after 250 ms). We report the MSE over time averaged over a 25 ms time-window. The constant model corresponds to the average MSE over time for a model that constantly predicts zero as displacement. **g** Mean squared error (MSE) during the auto-regression period for adLIF networks discretized with the Euler-Forward (brown) and Symplectic-Euler (pink) method on spring-mass systems with different frequency ranges.

frequency f , but not the decay rate r . The decay rate is given by $r = \sqrt{\alpha\beta}$ (see Eq. (55)) and is hence guaranteed to stay within the stability bound $r < 1$ for all $\tau_u > 0$ and $\tau_w > 0$, see Fig. 3d, right panel and Fig. 3e.

We analytically calculated stability bounds for both the Euler-Forward and SE discretization, see Methods Sections Derivation of stability bounds for EF-adLIF and Derivation of stability bounds for SE-adLIF. For each given tuple of time constants τ_u and τ_w we calculated a corresponding $a_{\max}$, that is, the maximum value of the parameter a for which the model is still stable.

This analysis shows that the SE discretization allows the neuron to utilize the full frequency bandwidth up to the Nyquist frequency at $\frac{1}{2\Delta t}$, at which aliasing occurs. Since we used a discretization time step of $\Delta t = 1$ ms for Fig. 3, the Nyquist frequency is 500 Hz. Theorem 1.1 below summarizes the full frequency coverage of SE-adLIF and the stability within this frequency range (see Methods, Section Proof of Theorem 1.1 for a proof).

Theorem 1.1. Let (τ_u, τ_w, a) be the parameters of an SE-adLIF neuron according to Eq. (11). For any frequency $f \in ([0, f_N])$ where $f_N = \frac{1}{2\Delta t}$ is the Nyquist frequency, and for any $\tau_u, \tau_w > 0$, there exists a unique parameter a such that the neuron has intrinsic frequency f . For any such parameter combination, the neuron in the sub-threshold regime is asymptotically stable with decay rate $r = \sqrt{\alpha\beta} < 1$ where $\alpha = e^{-\frac{\Delta t}{\tau_u}}$ and $\beta = e^{-\frac{\Delta t}{\tau_w}}$.

The upper frequency bound of adLIF neurons using the Euler-Forward discretization is illustrated in Fig. 3f. We can observe that for the Euler-Forward method, the maximum admissible frequency for stable dynamics converges toward zero as τ_u and τ_w increase (see

Section Stable ranges for intrinsic frequencies of EF-adLIF in Methods for proof).

An immediate advantage of using the SE-adLIF over EF-adLIF is the guaranteed stability over the entire range of possible oscillation frequencies. This property in particular comes into play when the neuron parameters τ_u , τ_w , a , and b are trained. While for most tasks only a sub-range of this viable frequency range might be required, it is guaranteed that the SE-adLIF neuron is stable for any such frequency. In other words, for SE-adLIF neurons with frequencies below the Nyquist frequency, no unstable parameter configurations exists. This is not the case for the EF-adLIF neuron: even in instances of very low oscillation frequencies the stability boundary might be overshoot (see Fig. 3c, d). Later in the manuscript (see Section Accurate prediction of dynamical system trajectories and Fig. 4g), we discuss the relationship between the neuron frequency range and the performance in an oscillator regression task. In our simulations, we clip a to fixed constant upper and lower bounds, given by task-dependent hyperparameters, independent of parameters τ_u and τ_w . While this constraint suffices for most tasks, it introduces a trade-off between the decay of the neuron and the oscillation frequency. This can be observed in Fig. 3c (right), where neurons with a high frequency are restricted to a fast decay. A trivial extension to the SE-adLIF model would be to dynamically adjust the upper bound for parameter a with respect to parameters τ_u and τ_w by computing $a_{\max}^{\text{SE}}$ (as defined by Eq. (68) in “Methods”) after each training step and clipping a to the interval $[0, a_{\max}^{\text{SE}}]$. This would allow the neuron model to utilize the entire frequency range for any combination of τ_u and τ_w . This extension is not possible for the EF-adLIF neuron, since using $a_{\max}^{\text{EF}}$ (as defined by Eq. (34) in “Methods”) as an

Table 1 | Comparison of recurrent LIF and adLIF networks on spike-encoded speech recognition datasets

	Model	Publication	#Params	Test Acc. [%]
SHD	LIF	Cramer et al. 2022 ³⁵	N.A.	83.2 ± 1.3
	LIF	Deckers et al. 2024 ²⁴	37.9k	84.49
	LIF	Bittar & Garner 2022 ¹⁶	141k	87.04
	LIF	Bittar & Garner 2022 ¹⁶	3.8M	89.29
	LIF	This work	450k	90.27 ± 0.73
	SE-adLIF	This work	450k	95.81 ± 0.56
SSC	LIF	Cramer et al. 2022 ³⁵	N.A.	50.9 ± 1.1
	LIF	Deckers et al. 2024 ²⁴	0.34M	71.76
	LIF	Bittar & Garner 2022 ¹⁶	141k	66.67
	LIF	Bittar & Garner 2022 ¹⁶	1.1M	68.14
	LIF	This work	1.7M	75.23
	SE-adLIF	This work	1.6M	80.44 ± 0.26
ECG	LIF	This work	1.8k	77.8 ± 1.85
	SE-adLIF	This work	1.8k	86.88 ± 0.40

The 'N.A.' indicates lack of information from the authors. 'This work' indicates our experimental results. The reported results correspond to the accuracy on the corresponding test set. Test accuracy is provided as mean ± SD when known.

upper bound instead of a constant value would still not allow the neuron to use the entire frequency range. This exact case, where a is set to $a_{\max}^{\text{EF}}$, is shown in Fig. 3f.

The favorable stability properties induced by the SE discretization should generalize well to other neuron models with two bi-directionally coupled neuron states. Two examples for such neuron models are the adaptive exponential integrate-and-fire (AdEx)³⁴ model and the Balanced Harmonic Resonate-and-Fire (BHRF)¹⁷ model. For the latter, we observed that applying the SE-discretization not only alleviates the necessity of the frequency-dependent divergence boundary, which was introduced by the authors to ensure stability of the model, but also recovers the direct relationship between neuronal parameters ω and b and the effective oscillation frequency ω_{eff} and effective damping coefficient b_{eff} of the discretized neuron. Details can be found in Supplementary Note 1 and Supplementary Fig. 1.

Improved performance of SE-discretized adaptive RSNNs

We first evaluated how recurrent networks of the described adLIF neurons perform in comparison to classical vanilla LIF networks. We compared LIF and SE-adLIF networks on two commonly used audio benchmark datasets: Spiking Heidelberg Digits (SHD)³⁵ and Spiking Speech Commands (SSC)³⁵, as well as an ECG dataset previously used to test ALIF neurons²⁵. For LIF baselines, we used results from previously reported studies as well as additional simulations to ensure identical setups and comparable parameter counts.

We obtained our results by constructing a recurrently connected SNN composed of one or two layers (depending on the task) of adLIF (resp. LIF) neurons, followed by a layer of leaky integrator (LI) neurons to provide a real-valued network output. We trained both the adLIF and LIF SNNs using BPTT with surrogate gradients^{7,8,13,36}. We used a dropout rate of 15% except noted otherwise, but otherwise no regularization, normalization, or data augmentation methods. The trained parameters included the synaptic weights θ , as well as all neuron parameters a , b , τ_u and τ_w in the SE-adLIF case, and membrane time constants τ in the LIF case. The parameters were not shared across neurons such that each neuron could have individual parameter values. Neurons were initialized heterogeneously, such that for each neuron the initial values of these parameters were chosen randomly from a uniform distribution over a pre-defined range. Heterogeneity has previously been shown to improve the performance of SNNs³⁷. We applied a reparametrization technique for the training of time constants τ_u and τ_w and parameters a and b for

the SE-adLIF model, as well as the membrane time constants τ for the LIF models, see Methods for details.

Table 1 summarizes both the baselines from prior studies and our own results. While LIF networks in our simulations performed better than all previously reported LIF baselines, they still performed significantly worse than the SE-adLIF networks with the same or lesser parameter counts across all tasks. This result provides clear empirical support for the superiority of adLIF networks over LIF networks, both in terms of parameter efficiency and overall performance.

In the previous sections, we discussed the theoretical advantages of SE discretization over the more commonly used Euler-Forward discretization of the adLIF neuron model. The discussed theoretical advantages of SE, for example near-independence of the neuron dynamics from the discretization time step Δt or the closer adherence to the continuous model, give indirect practical benefits when dealing with such neurons. It is not clear however, whether the utilization of the SE discretization can provide an improvement in performance over the commonly used EF method. To answer this question, we not only compared both the EF-adLIF and the SE-adLIF model to each other, but also to state-of-the-art spiking neural networks: a model with threshold adaptation (ALIF)²⁵, a constrained variant of the adLIF neuron model, similar to the model in this study, but with differences in discretization and neuron formulation (cAdLIF)²⁴, another adLIF network but with batch normalization (RadLIF)¹⁶, a feed-forward model with delays implemented as temporal convolutions (DCLS-Delays)³⁸, and the balanced resonate-and-fire neuron model (BHRF)¹⁷, a variant of the resonate-and-fire neuron¹² where output spikes do not disrupt the phase of the membrane potential oscillation.

In Table 2 we report the test accuracy of the various models on the corresponding test sets. SHD does not define a dedicated validation set and previous work reported performances for networks validated on the test set, which is methodologically not clean. We therefore report results for two validation variants for SHD: with validation on the test set (to ensure comparability) and with validation on a fraction of the training set.

For all considered datasets, recurrent SE-discretized adLIF networks performed better than previously considered recurrent SNNs. For SSC, their performance was slightly below that of the DCLS model³⁸, a feed-forward network using extensive delays trained via dilated convolutions. Unlike our model, however, DCLS employs temporal convolutions to implement delays and incorporates batch normalization. These properties make the DCLS model less suitable for neuromorphic use cases. Nevertheless, we included it in our results table for comparison, as the

Table 2 | Comparison of accuracies of different models on the test sets of SHD, SSC and ECG

	Model	Rec.	#Params	#Runs	Test Acc. [%]
SHD	LIF	✓	0.45M	10	90.27 ± 0.73
	ALIF ²⁵	✓	0.14M	1	90.4
	cAdLIF ²⁴	✗	38.7k	10	94.19
	RadLIF ¹⁶	✓	3.9M	-	94.62
	DCLS-Delays ³⁸	✗	0.2M	10	95.07 ± 0.24
	EF-adLIF (2 layers)	✓	0.45M	20	94.68 ± 0.57
	SE-adLIF (1 layer)	✓	37.5k	20	94.59 ± 0.27
	SE-adLIF (2 layers)	✓	0.45M	20	95.81 ± 0.56
	BHRF ¹⁷	✓	0.1M	5	92.7 ± 0.7
	SE-adLIF (1 layer)	✓	37.5k	20	93.18 ± 0.74
SHD*	SE-adLIF (2 layers)	✓	0.45M	20	93.79 ± 0.76
	LIF	✓	1.7M	1	75.23
	ALIF ²⁵	✓	0.78M	1	74.2
	RadLIF ¹⁶	✓	3.9M	-	77.4
	cAdLIF ²⁴	✗	0.35M	1	77.5
	DCLS-Delays ³⁸	✗	2.5M	5	80.69 ± 0.21
	EF-adLIF (2 layers)	✓	1.6M	5	79.80 ± 0.18
	SE-adLIF (2 layers)	✓	1.6M	5	80.44 ± 0.26
	ECG LIF (1 layer)	✓	1.8k	5	77.8 ± 1.85
	ALIF ²⁵	✓	1.8k	1	85.9
SSC	BHRF ¹⁷	✓	1.8k	5	87.0 ± 0.4
	EF-adLIF (1 layer)	✓	1.8k	20	87.10 ± 0.46
	SE-adLIF (1 layer)	✓	1.8k	20	86.88 ± 0.40
	EF-adLIF (2 layers)	✓	4.6k	20	87.83 ± 0.51
	SE-adLIF (2 layers)	✓	4.6k	20	88.18 ± 0.36
	ECG LIF (1 layer)	✓	1.8k	5	77.8 ± 1.85
	ALIF ²⁵	✓	1.8k	1	85.9
ECG	BHRF ¹⁷	✓	1.8k	5	87.0 ± 0.4
	EF-adLIF (1 layer)	✓	1.8k	20	87.10 ± 0.46
	SE-adLIF (1 layer)	✓	1.8k	20	86.88 ± 0.40
	EF-adLIF (2 layers)	✓	4.6k	20	87.83 ± 0.51
	SE-adLIF (2 layers)	✓	4.6k	20	88.18 ± 0.36
	ECG LIF (1 layer)	✓	1.8k	5	77.8 ± 1.85
	ALIF ²⁵	✓	1.8k	1	85.9

The asterisk (*) of SHD* denotes that a fraction of the training data was used as a validation set. BHRF used 5%, whereas we used 20% of the training data for validation. We report the test accuracy for the training epoch with the highest validation accuracy. For our model, SE-adLIF, we report the mean and standard deviation on 20 different random initializations. Rec.: model contains recurrent synaptic connections; #Runs: number of runs with different random seeds to obtain the results. Test accuracy is provided as mean ± SD.

delays in neural connectivity provide an interesting orthogonal complement to the enhanced somatic dynamics of the models studied in our work (see also Discussion). Networks composed of SE-discretized adLIF neurons (SE-adLIF networks) performed significantly better than those based on Euler-forward discretization (EF-adLIF networks) on SHD and SSC (significance values for a two-tailed t test were $p < 0.000001$ for SHD and $p < 0.005$ for SSC). Small networks with a single recurrent layer on ECG performed on-par ($p = 0.115$), while SE-adLIF networks significantly improved over EF-adLIF networks when larger networks with two recurrent layers were used ($p < 0.02$). We found that EF-adLIF networks suffered from severe instabilities if neuron parameters were not constrained to values in which the decay rate exceeds the critical boundary of $r = 1$, resulting in instabilities for example in the ECG task, see Supplementary Note 2 and Supplementary Table 1. The SE method is hence the preferred choice when adLIF neurons are used in a discretized form.

AdLIF neurons could, depending on their parameters, exhibit many different experimentally observed neuronal dynamics⁹. We wondered whether networks trained on spatio-temporal classification tasks utilized the diverse dynamical behaviors of adLIF neurons. To that end, we investigated the resulting parameterizations of adLIF neurons in networks trained on SHD, and indeed found a heterogeneous landscape of neuron parameterizations, see Supplementary Note 3 and Supplementary Fig. 2.

Accurate prediction of dynamical system trajectories

The benchmark tasks considered above were restricted to classification problems where the network was required to predict a class label. We next asked whether the rich neuron dynamics of adaptive neurons could be utilized in a generative mode where the network has to produce complex time-varying dynamical patterns. To that end, we considered a task in which networks had to generate the dynamics of a system of 4 masses, interconnected by springs with different spring constants, see Fig. 4a. Each training sequence consisted of the masses' trajectory over time for 500 ms (Fig. 4b) from a randomly sampled initial condition of this 4-degree-of-freedom dynamical system, where the displacement x_i of each mass i was encoded via a real-valued input current. During the first half of the sequence, the model was trained to produce single-step predictions, that is, it received the mass displacements $\mathbf{x}[k] \in \mathbb{R}^4$ as input at each time step k and had to predict the displacements $\mathbf{x}[k+1]$. In the second half, the model auto-regressed, i.e. it used its own prediction $\hat{\mathbf{x}}[k]$ to predict the next state $\mathbf{x}[k+1]$ (see Fig. 4c). Through this second phase, we tested if the network was able to accurately maintain a stable representation of the evolving system by measuring the deviation from the ground truth over time.

Note that in the spring-mass system the states are described by the displacement and velocity of the masses but only displacement information $\mathbf{x}[k]$ was available to the network. Hence, it is impossible to accurately predict the displacements of the masses at time $k+1$ from the displacements at time k alone. The network must therefore learn to keep track of the longer-time dynamics of the system.

In Fig. 4d, we show the ground truth of displacement x_1 for mass 1, as well as the prediction of the displacement by an SE-adLIF network with a single hidden layer of 200 neurons and a single-layer LIF network with the same number of parameters. After time step $t = 250$ ms, the auto-regression phase starts. These plots exemplify how the LIF network roughly followed the dynamics during the one-step prediction phase, but gradually diverged from the target in the auto-regression phase. In contrast, the generated trajectory of the SE-adLIF network stayed close to the ground truth system throughout the auto-regression phase. Fig. 4e shows the mean squared error (MSE) of several models and model sizes during this autoregressive phase. SE-adLIF networks consistently outperformed LIF networks as well as non-spiking long-short-term memory (LSTM) networks (note the log-scale of the y-axis). Moreover, we observed that their performance scaled better with network size (1.7 and 1.3 MSE improvement factor per doubling of the network size for SE-adLIF and LIF networks respectively). Figure 4f shows how fast the models degrade towards the baseline of a model that constantly outputs zero. We observe that small SE-adLIF networks with 200 neurons approximated the trajectory of the dynamical system in the auto-regression phase for a much longer duration than the best LIF network with 3200 neurons. Interestingly, when we trained SE-adLIF networks without recurrent connections, their dynamics degraded clearly slower than LIF networks with recurrent connections (Supplementary Fig. 3), which underlines the utility of the inductive bias of oscillatory neurons for such generative tasks.

Additionally, we used this setup to compare the Symplectic-Euler discretization (SE-adLIF networks) with the Euler-Forward discretization (EF-adLIF networks). Since in this task the frequency bandwidth can be controlled directly via the spring coefficient, we generated

Table 3 | Performance comparison between LIF, EF-adLIF, and SE-adLIF networks with state-of-the-art audio codecs in the audio compression task

Model	Sam. Rate	#Params	Avg. Bandw.	SI-SNR ⁴⁸	VISQOL ⁴⁹
LIF	24 kHz	543k	4.6 kSp./s	6.90 ± 0.05	3.46 ± 0.01
EF-adLIF	24 kHz	543k	5.0 kSp./s	7.80 ± 0.04	3.82 ± 0.01
SE-adLIF	24 kHz	543k	5.2 kSp./s	8.39 ± 0.05	3.91 ± 0.004
RVQ ⁴⁷	24 kHz	14.8M	6 kbps	4.14	4.38
OPUS ⁴⁵	24 kHz	-	6 kbps	1.35	3.69
EVS ⁴⁶	16 kHz	-	6 kbps	5.51	4.11

We used the same network layout for all neuron models and report on the LibriTTS clean test set⁷⁹ across 4 runs except for LIF models, for which only 2 runs converged and were taken into account. See Section Details for the audio compression task in Methods for details. SI-SNR denotes the scale-invariant signal-to-noise ratio⁴⁸, VISQOL the value obtained from the Virtual Speech Quality Objective Listener⁴⁹. OPUS and EVS are audio codecs, hence no parameter counts are provided. Sam. Rate refers to the respective sampling rate of the audio signal. Avg. Bandw. refers to the average number of spikes per second (Sp./s) at the bottleneck layer for the spiking networks, and the data transmission rate for the other, frame-based methods. Test accuracy is provided as mean ± SD.

spring-mass systems of increasing maximal frequency. We trained EF-adLIF networks and SE-adLIF networks under the same range of time-constants (τ_u and τ_w) and a restricted range for the adaptation parameter a . For Euler-Forward, a was restricted between 0 and $a_{\max}$, where $a_{\max}$ is the maximal parameter value for a that is stable under this discretization, resulting in a [0, 30] Hz range of frequencies that can be represented by the neurons for the chosen range of time-constants. For Symplectic-Euler, all frequencies below the Nyquist frequency are stable, so we simply choose $a_{\max}$ to achieve a frequency range of [0, 60] Hz. The results are shown in Fig. 4g. As expected, the two methods have similar performance at low frequencies. For dynamics with a larger frequency bandwidth however, EF-adLIF networks performed significantly worse. Additionally, the increased variance of the error indicates stability problems. These experimental results support our claim that the wider stability region of the SE-adLIF network allows the model to converge over a wide range of data frequencies.

High-fidelity neuromorphic audio compression

In all experiments so far we observed a significant superiority of the adLIF neuron over the LIF neuron, both in terms of parameter efficiency and overall performance. Yet, it is unclear how these observed improvements transfer from benchmarks and toy tasks to real-world neuromorphic applications. To take a step towards answering this question, we compared the performance of adLIF and LIF neurons in the task of raw audio compression. The goal of this task is to first compress and then transmit a raw audio signal as energy-efficient as possible, while sacrificing as little signal quality as possible. Similar setups with neuromorphic processing and spike-based transmission have been discussed in various studies^{39–43} as promising research direction for low-power IoT applications with smart wireless sensors. Our study addresses audio compression using plain RSNNs that deliberately exclude batch normalization, temporal convolutions, and transformer-based architectures to maintain compatibility with standard neuromorphic processors.

We conceptually consider a small device containing a digital neuromorphic chip that receives raw, unprocessed audio from a microphone. For our study, we used SNN simulations and did not implement this setup in real hardware, but simulated the SNNs that would run on such chips (see Supplementary Fig. 4a for a schematic illustration). In the conceptual setup, the chip processes the waveform by implementing a small SNN with a bottleneck output layer consisting of very few neurons. It sends the spike-encoded audio data through a sparse wireless communication channel⁴³ to a receiving device. This receiving device, a second neuromorphic processor, could in principle perform arbitrary post-processing on the spike-encoded data. In our simulations we considered the most general case, which is the reconstruction of the ground truth waveform from the sparse spikes. For the spike-based communication we assume low-latency pulse-driven radio transmission, for example IR-UWB⁴⁴, that features adaptive energy

consumption, depending on the presence of input signal. In the absence of an input signal (i.e. silence), almost no energy is consumed by the transmitting device, hence this technology is a promising candidate for ultra low-power neuromorphic sensing devices⁴². In contrast, if conventional frame-based digital transmission is used, the transmission rate is constant and power is consumed at a constant rate. In pulse-driven spike encoding, the timing of spikes is implicitly encoded by the timing of the emitted radio pulse, such that the spike timing does not need to be explicitly transmitted as payload.

Audio compression requires balancing the quality of the reconstructed signal against the data transmission rate at the bottleneck. In our simulations, we constrained the encoder SNN to very sparse spiking activity at the output layer to achieve low-bandwidth transmission. This aligns with our goal of ultra-low-energy processing, as energy consumption in neuromorphic systems is directly tied to spike rate. We considered as few as 16 output neurons for the encoder, regularized to not exceed a total maximum spike rate of 6k spikes per second. Assuming that a single spike is equivalent to 1 bit, the upper regularization bound for the average data transmission rate between encoder and decoder is 6 kbps. Under this constraint, we compared the quality of the reconstructed audio signal of LIF and adLIF neurons with common audio compression codecs^{45,46} and the state-of-the-art Residual Vector Quantization (RVQ) method⁴⁷ evaluated at the same bandwidth of 6 kbps. The results are shown in Table 3. Details on the task setup and the simulations can be found in Section Details for the audio compression task in Methods. We provide uncompressed audio samples (Supplementary Audio 1) and reconstructions using the LIF (Supplementary Audio 2), EF-adLIF (Supplementary Audio 3), and SE-adLIF networks (Supplementary Audio 4) as supplements to this article. Our findings indicate that adaptive LIF neurons achieve a significantly higher reconstruction quality than vanilla LIF neurons (see Supplementary Fig. 4b for an example). Moreover, SE-discretized adLIF neurons yield significantly higher reconstruction quality, as demonstrated by both quantitative metrics (Table 3) and visual waveform comparisons (Supplementary Fig. 4b). Given that waveform data spans a broad frequency spectrum, the stability of the SE-adLIF neurons over the entire frequency range – as discussed in Section Stability analysis of discretized adLIF models – offers a clear advantage over EF-adLIF. Since LIF neurons lack intrinsic membrane potential oscillations, they depend heavily on recurrent network dynamics to detect, encode and decode oscillatory patterns in their input. We observed the same phenomenon in the oscillatory task in Fig. 4, where LIF neurons performed poorly. While the SE-adLIF model achieved the best performance in terms of scale-invariant signal-to-noise ratio⁴⁸, the state-of-the-art neural network model RVQ exhibits better performance on the VISQOL measure⁴⁹, but at the cost of a much larger model with a >27× increase in the number of parameters. The compactness of the SE-adLIF model allowed audio compression and decoding in 1.3× real-time on a consumer single tread CPU (AMD Ryzen 7 5800H). In Table 3, we also report performances of

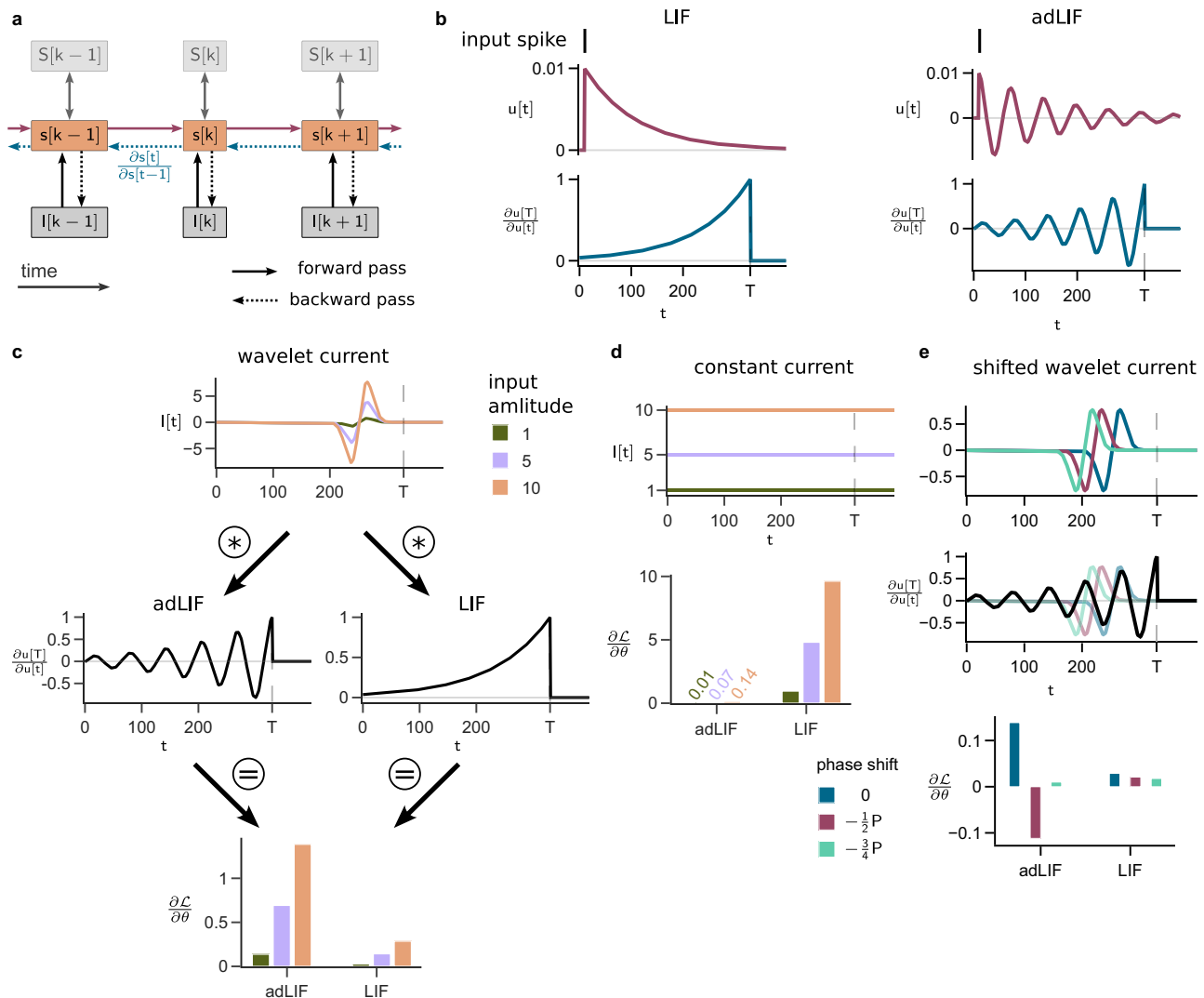


Fig. 5 | Inductive bias of adLIF gradients. **a** Computational graph showing the state-to-state derivative $\frac{\partial s[t]}{\partial s[t-1]}$ back-propagating through time. $s[k]$ denotes the state vector (Eq. (5)). **b** Response of the membrane potential of a LIF neuron (left) and an adLIF neuron (right) to a single input spike. The shape of the derivative $\frac{\partial u[T]}{\partial u[t]}$ matches the reversed impulse response function. **c** Comparison of the derivative $\frac{\partial \mathcal{L}}{\partial \theta}$ for a wavelet input current. The multiplication from Eq. (12) of the input current with the state derivative is schematically illustrated for both, the adLIF and the LIF case. The frequency of the wavelet approximately matches the

intrinsic frequency of the oscillation of the membrane potential oscillation of the adLIF neuron. The bar plot on the bottom shows the derivative $\frac{\partial \mathcal{L}}{\partial \theta}$ for both neurons, where color indicates input amplitude. **d** Same as panel c but for a constant input current. **e** Same as panel c but for different positions of the wavelet current. Middle plot shows the alignment between input and back-propagating derivative $\frac{\partial u[T]}{\partial u[t]}$ for the adLIF neuron. Input wavelet is given with a phase shift of $0, -\frac{1}{2}P$ and $-\frac{3}{4}P$ with respect to the period P of the adLIF neuron oscillation.

two standard audio codecs (OPUS and EVS) on our test set, showing that the SE-adLIF networks achieve competitive performance. In the next few sections, we further explore the inductive bias introduced by oscillatory membrane potentials.

Computational properties of adLIF networks

Our empirical results above demonstrate the superiority of oscillatory neuron dynamics over pure leaky integration in spiking neural networks, which is in line with prior studies^{13–17}. In the following, we analyze the reasons behind this superiority.

Adaptation provides an inductive bias for temporal feature detection. In gradient-based training of neural networks, the gradient determines to which features of the input a network ‘tunes’ to. Hence, understanding how gradients depend on certain input features contributes to the understanding of network learning dynamics. When a recurrent SNN is trained with BPTT, the gradient propagates through

the network via two different pathways: the recurrent synaptic connections and the implicit neuron-internal recurrence of the neuron state $s[k]$.

For the following analysis, we ignored the explicit recurrent synaptic connections and focused on how the backward gradient of the neuron state determines the magnitude of weight updates in different input scenarios, see Fig. 5a.

Consider the derivative $\frac{\partial u[T]}{\partial u[k]}$ of the membrane potential at a time step T with respect to the membrane potential at some prior time step k . Intuitively, this derivative indicates how small perturbations of $u[k]$ influence $u[T]$. Figure 5b shows this derivative for a LIF neuron (left) and an adLIF neuron (right). Because this derivative is the reverse of the model’s forward impulse response, it exhibits oscillations in the case of the adLIF neuron and reversed leaky integration for a LIF neuron. Consider a LIF or an adLIF neuron with a single synapse with weight θ and input $I[k]$. A loss signal $\frac{\partial \mathcal{L}}{\partial u[T]}$ (set to 1 in our illustrative example) is provided at time-step T . The resulting gradient $\frac{\partial \mathcal{L}}{\partial \theta}$, used to

compute the update of synaptic weight θ , is given by

$$\frac{\partial L}{\partial \theta} \propto \frac{\partial L}{\partial u[T]} \sum_{k=1}^T \frac{\partial u[T]}{\partial u[k]} I[k]. \quad (12)$$

This equation makes explicit that the weight change is proportional to the correlation between the input currents $I(T)$, $I(T-1)$, $I(T-2)$, ... and the internal derivatives $\frac{\partial u[T]}{\partial u[T-1]}$, $\frac{\partial u[T]}{\partial u[T-2]}$, $\frac{\partial u[T]}{\partial u[T-3]}$, ...

This is illustrated in Fig. 5c for temporal input currents—realized as wavelets—at different amplitudes. The magnitudes of the resulting gradients for the LIF and adLIF model are quite complementary.

The correlation between the wavelet and the oscillations of the adLIF neuron's state-derivative results in a strongly amplitude-dependent gradient. In contrast, the leaky integration of the LIF neuron averages the positive and negative region in the input wave. The situation changes drastically for a constant input current, see Fig. 5d. For LIF neurons, the gradient $\frac{\partial L}{\partial \theta}$ strongly increases with increasing input current magnitude in this scenario. In contrast, the gradient of the adLIF neuron only weakly depends on the magnitude of the constant input current, in fact the gradient is nearly non-existent. This can be explained by the balance between positive and negative regions of the adLIF gradient (compare with Fig. 5b), resulting in almost zero if multiplied with a constant and summed over time.

The temporal sensitivity of the adLIF gradient is even more evident when we consider the gradients for different positions of a wavelet current (Fig. 5e). The sign and magnitude of the gradient strongly depend on the position of the wavelet for the adLIF neuron, but not for the LIF neuron. If the wavelet input is aligned with the oscillation of the back-propagating derivative, the resulting gradient is strongly positive. For a half-period ($-\frac{1}{2}P$) phase shift, the gradient is negative and for a $-\frac{3}{4}P$ phase shift, the resulting gradient is low in magnitude due to misalignment of oscillation and input. This gradient encourages the neuron to detect temporal features in the input, that is, temporally local changes in the input, either as changes in the spike rate (e.g. Fig. 2d) or in the input current (e.g. Fig. 5c, e) with specific timing. This sensitivity hence provides an inductive bias for spatio-temporal sequence processing tasks.

Networks of adLIF neurons tune to high-fidelity temporal features.

Through the rich dynamics and the consequential inductive bias towards learning temporal structure in the input, adLIF neurons should be well-suited for tasks in which spatio-temporal feature extraction is necessary. In order to investigate how well temporal input structure can be exploited by networks of adLIF neurons as compared to networks of LIF neurons, we considered a conceptual task that can be viewed as prototypical temporal pattern detection. We refer to this task as the burst sequence detection (BSD) task.

In the BSD task, a network has to classify temporal patterns of bursts from a population of n input neurons, see Fig. 6a.

This task is motivated from neuroscientific experiments which show the importance of sequences of transient increases of spike rates in cortex^{50,51}.

A class in this task is defined by a specific pre-defined temporal sequence of bursts across a fixed sub-population of three of these neurons. Other neurons emit a random burst at a random time each and additionally, neurons fire with a background rate of 50 Hz. Bursts were implemented as smooth, transient increases of firing rate resulting in approximately 7 spikes per burst, see Section Details for the Burst Sequence Detection (BSD) task in Methods for details. We tested single-layer recurrent adLIF networks (510 neurons) and single-layer recurrent LIF networks with the same number of trainable parameters. AdLIF networks clearly outperformed LIF networks on this task, see Fig. 6b. For the case of 10 classes, the adLIF network reached

an average classification error of 2.31% on the test set, whereas the LIF network only achieved a test error of 6.96% despite a low training error (<2%).

In order to evaluate to what extent the computations of the SNNs considered relied on temporal features of the input, we used a technique commonly applied to artificial neural networks that allows to visualize the input features that cause the network to predict a certain class⁵².

The idea of this optimization-based feature visualization procedure is to generate an artificial data sample X^* that maximally drives the network output towards a pre-defined target class c :

$$X^* = \arg \min_X L(X, c), \quad (13)$$

where $L(X, c)$ denotes the loss of the network output for input X and target class c . In order to estimate X^* , one starts with a uniform noise input X^0 and updates the input using gradient descent to minimize the loss, i.e., the input X^{k+1} after update $k+1$ is given by

$$X^{k+1} = X^k - \frac{\eta}{\zeta_k} \nabla_X L(X, c)|_{X^k}, \quad (14)$$

where η is the update step size, $\nabla_X f(X)|_{X^k}$ denotes the gradient of f with respect to X evaluated at X^k , and ζ_k is a normalization factor, see Fig. 6c and Section Optimization-based feature visualization in Methods for details. After each update, we applied additional regularization to the data sample X^{k+1} (see Methods for details). We repeated this procedure for $K = 400$ iterations, such that the final X^k yielded a very strong prediction for class c .

We performed this feature visualization for the BSD task and for the SHD task. Figure 6d shows the resulting artificial samples X^k for an adLIF network and a LIF network trained on the BSD task. The class-defining burst timings are indicated as black circles with white filling.

The adLIF-generated samples exhibited a strong temporal structure that captures the relevant temporal structure of the class. This can be observed visually by comparing the position of strong activations with the class-defining burst timings. In contrast, the samples generated from the trained LIF network displayed less precise resemblance of class-descriptive features, and showed less temporal variation. This gap of specificity of the features in the generated samples of adLIF versus LIF networks might explain the performance gap between these: While the less precise temporal tuning of LIF networks suffices to achieve a high accuracy on the training data, it falls behind in terms of generalization on the test set, due to confusion of temporal features from different classes. Interestingly, the same analysis for an ALIF network²⁵ with threshold adaptation revealed that the temporal tuning of ALIF networks is comparable to that of LIF networks, indicating the importance of oscillatory dynamics for temporal feature detection, see Supplementary Note 4 and Supplementary Fig. 5.

Similar results were obtained for SHD, Fig. 6e, where we applied $K = 200$ iterations. The underlying class-descriptive features in the SHD task are less clearly visible, since it is instantiated from natural speech recordings. Nevertheless, one can clearly observe richer temporal structure of the adLIF-generated samples. These samples indicate, that the two network models tune to very different features of the input, which is again in alignment with the above reported inductive bias of the gradient. While LIF networks tend to tune to certain spike rates of different input neurons over prolonged durations, adLIF neurons rather tune to local variations of the spike rates. In summary, our analysis supports the hypothesis that the superior performance of SNNs based on adLIF neurons on datasets like SHD stems from the fact that temporal features can effectively be learned and detected.

Inherent normalization properties of adLIF neurons. Artificial neural networks as well as spiking neural networks are often trained using

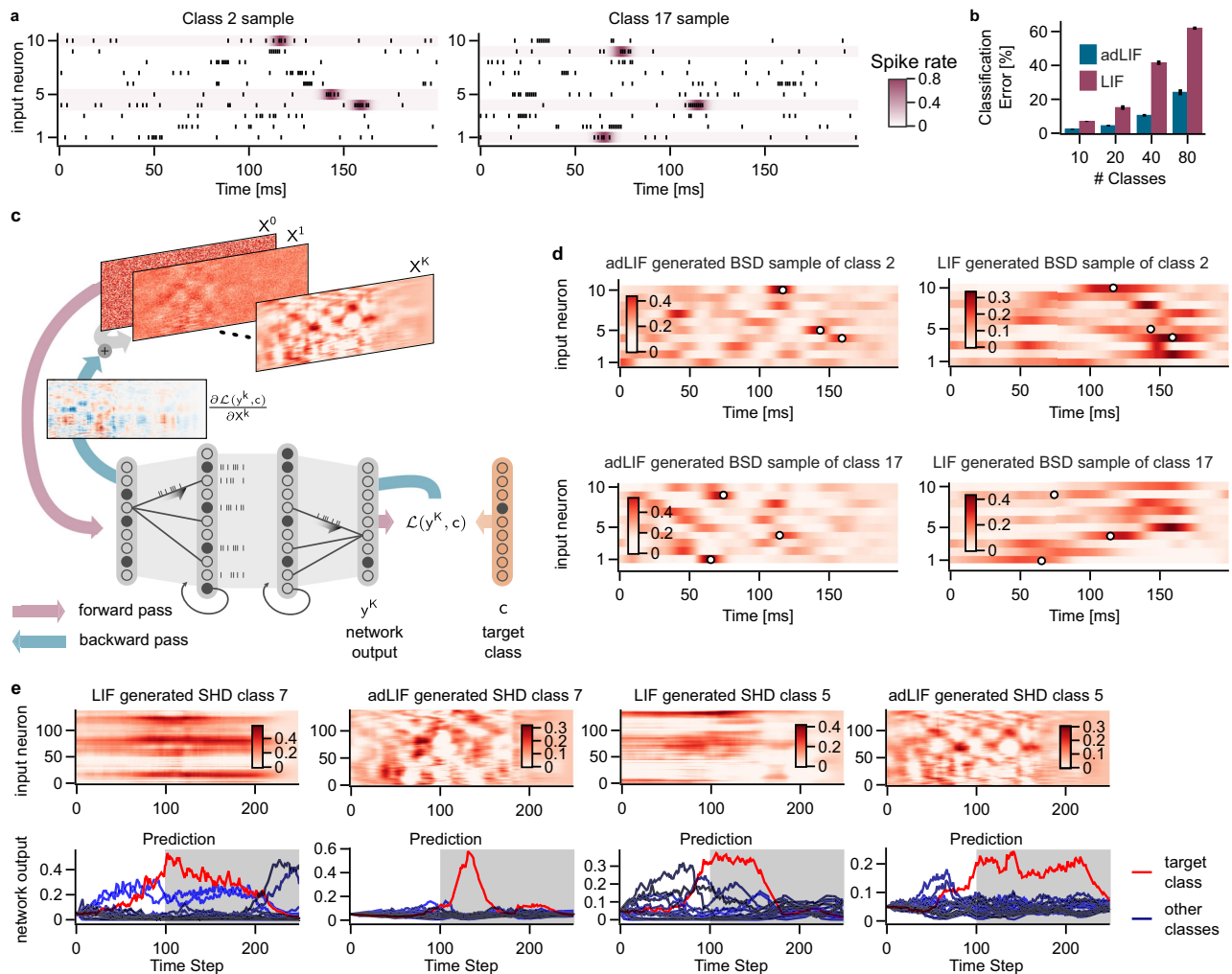


Fig. 6 | Temporal feature detection in adLIF networks. **a** Two samples of classes 2 and 17 of the burst sequence detection (BSD) task (see main text). **b** Classification error of adLIF and LIF networks with equal parameter count for different numbers of classes in the BSD task. **c** Schematic illustration of network feature visualization. An initial noise sample x_0 is passed through a trained network with frozen network parameters. The classification loss of the network output with respect to some predefined target class c is computed and back-propagated through the network to obtain the gradient $\nabla_x \mathcal{L}(X, c)|_{x_0}$ of the loss with respect to input x^0 . This gradient is applied to the sample and the procedure is repeated to obtain a final sample x^k after

$K = 400$ iterations. **d** Samples generated by the feature visualization procedure from panel c from networks trained on the 20-class BSD task. White dots denote the locations of the class-descriptive bursts. We generated samples for classes 2 and 17 which were the most misclassified classes of LIF and adLIF networks respectively. **e** Samples generated from an adLIF network and a LIF network trained on SHD for different target classes c (top) and the corresponding network output over time (bottom). The gray shaded area (at $t > 100$) denotes the time span relevant for the loss, all outputs before this time span were ignored, see “Methods” for details.

normalization techniques that normalize the input to the network layers, for example along the spatial dimension of a single batch⁵³. As inputs to SNNs are in general temporal sequences, normalizations over both the spatial and temporal dimensions have been introduced^{18–21}. Such normalization techniques, in particular over the temporal dimension, are however problematic from an implementation perspective, especially when such networks are deployed on neuromorphic hardware and the entire sequence is not known upfront.

In contrast, all results reported in this article have been achieved without an explicit normalization technique, indicating that normalization over the temporal dimension is not necessary for networks of adLIF neurons.

We argue that good performance without normalization is possible due to the negative feedback loop through the adaption current in adLIF neurons (Eqs. (2), (3)) which inherently stabilizes neuron responses as long as neurons are in the stable regime. In addition, the oscillatory sub-threshold response (Fig. 1b) tends to filter out constant offsets in the forward pass. Similarly, the oscillating gradient (Fig. 5b)

tends to filter out constant activation offsets during training, thus stabilizing training.

To test the stabilizing effect of adLIF neurons, we investigated how a constant offset in the input during test time effects the accuracy of RSNNs. To this end, we tested LIF and adLIF networks trained on the clean SHD dataset (Table 2) on biased SHD test examples which we obtained by adding a constant offset to each input dimension. More precisely, the biased sample $\hat{x}[k]$ at time step k was given by $\hat{x}[k] = x[k] + \kappa \bar{x}$, where $\bar{x}$ is the mean over all input dimensions and time steps and $\kappa \geq 0$ scales the bias strength.

The classification accuracy of the network on these biased test samples as a function of the bias coefficient κ is shown in Supplementary Fig. 6a. Surprisingly, even for large bias values of $\kappa = 1$ the adLIF network maintained a high classification accuracy (>77%). In comparison, the accuracy of a LIF network dropped rapidly below 10% for $\kappa > 0.6$.

In a second experiment, instead of a constant bias, we added background noise via random spikes to the raw input data. Again, even

for high background noise rates of 20 Hz, the adLIF network was surprisingly robust, maintaining an accuracy of >82%. In contrast, the LIF network accuracy for this noise level dropped to <18%, see Supplementary Fig. 6b,c. These results support the claim that normalization methods are not necessary to train noise-robust high-performance adLIF networks, which represents a substantial advantage of this model over LIF-based SNNs in neuromorphic applications.

Discussion

Spiking neural network models are the basis of many neuromorphic systems. For a long time, these networks used leaky integrate-and-fire (LIF) neurons as their fundamental computational units. More recent work has shown that networks of adaptive spiking neurons outperform LIF networks in spatio-temporal processing tasks. However, a deep understanding of the mechanisms that underlie their superiority was lacking. In this article, we investigated the underpinnings of the computational capabilities of networks of adaptive spiking neurons.

We first demonstrated, both analytically and empirically, why the commonly used Euler-Forward (EF) discretization is problematic for multi-state neuron models. Specifically, we showed that the EF-discretization leads to substantial deviations in the neuron model dynamics compared to its continuous counterpart (see Fig. 3c) and that the magnitude of these deviations strongly depends on the discretization time step (Fig. 3a, b). Moreover, parameter configurations that yield stable dynamics in the continuous model often result in divergent dynamics when applied to the EF-discretized model. Another unintended consequence of EF discretization is the introduction of interdependencies between parameters that are otherwise decoupled. We briefly examine this effect on the Balanced Harmonic Resonate-and-Fire (BHRF) model¹⁷ in Supplementary Note 1 and Supplementary Fig. 1. Finally, we demonstrated that all of these drawbacks can be eliminated, without incurring additional computational cost, by using the Symplectic Euler discretization instead of EF.

Many digital neuromorphic chips implement time-discretized SNNs^{3,4,29,54}. Adaptive neurons are an attractive model for such systems as they only add a single additional state variable per neuron. As synaptic connections are typically dominating implementation costs, this approximate doubling of resources needed for neuron dynamics is well justified by the improved performance. For example, Fig. 4f shows that there exist tasks where adLIF networks can achieve superior performance to LIF networks with orders of magnitudes less parameters. Compared to the standard Euler-discretized adLIF model, the computational operations needed to implement the SE-adLIF model are identical. Hence, the improved stability properties of this model are practically for free.

Our analysis in Section Computational properties of adLIF networks indicates that adLIF should be well-suited to learn relevant temporal features from input sequences. Interestingly, our investigations revealed that LIF networks are surprisingly weak in that respect. This is witnessed by its low performance at the burst sequence detection task (Fig. 6) as well as by our input-feature analysis of trained LIF networks (Fig. 6 panels d and f). This is surprising, as theoretical arguments suggest that SNNs in general should be efficient in temporal computing tasks⁴. Our analysis suggests that the gradients in LIF networks fail to detect such temporal features, while for adLIF networks, these gradients are actually biased towards those. Our empirical evaluation supports this claim as adLIF networks excel at the burst sequence detection task (Fig. 6), a task which we designed specifically to test temporal feature detection capabilities of SNNs (Fig. 6). The input-feature analysis for trained adLIF networks further supports this view (Fig. 6 panels b and d).

The auto-regressive task on a complex spring-mass system (Fig. 4) can also be seen as a conceptual task designed to investigate the capabilities of SNNs to predict the behavior of complex oscillatory systems. Note that, although the trajectories of masses that have to be predicted

are periodic, the period is very long due to the complex interactions of the four masses. This conceptual task is of high relevance as oscillations are ubiquitous in physical systems and biology, for example in limb movement patterns^{55,56}. The superiority of adLIF networks in this task can be explained by the principles of physics-informed neural networks⁵⁷. In this framework, underlying physical laws of some training data are molded into the architecture of neural networks, such that the functions learned by these networks naturally follow these laws. Obviously, the oscillatory sub-threshold behavior of adLIF neurons fits well to the oscillatory dynamics of the spring-mass system. Taking one step further, we showed that the observed advantage of adaptive LIF neurons in this oscillatory toy task successfully transfers to the more complex, real-world inspired task of audio compression (see Supplementary Fig. 4), providing a promising perspective for adaptive neurons in neuromorphic applications. Interestingly, we observed that the LIF networks were particularly sensitive to the choice of hyperparameters in this task in contrast to the SE-adLIF networks, due to the long sequence length of 2,560 time steps.

Finally, we empirically demonstrated the robustness of adLIF networks towards perturbations in the input, showcasing their invariance to shifts in the mean input strength. Surprisingly, the accuracy of adLIF networks on the Spiking Heidelberg Digits (SHD) dataset maintained a high level (>80%) even after doubling the mean input strength, whereas LIF networks drop to an accuracy of 10% already for a much lower increase. We argue that this inheritance alleviates the need for layer normalization techniques in other types of artificial and spiking neural networks. Indeed, all our results were achieved without explicit normalization techniques. This finding is in particular relevant for neuromorphic implementations of SNNs, as explicit normalization is hard to implement in neuromorphic hardware, in particular for recurrent SNNs.

Oscillatory neural network dynamics have been studied not only in the context of SNNs. Several works^{33,58,59} identified favorable properties adopted by models utilizing some form of oscillations. Rusch et al.⁵⁸ for example studied an RNN architecture in which recurrent dynamics were given by the equation of motion of a damped harmonic oscillator. The authors found, that these oscillations not only alleviate the vanishing and exploding gradient problem⁶⁰, but also perform well on a large variety of benchmarks. Effenberger et al.³³ proposed oscillating networks as a model for cortical columns. In the field of artificial neural networks, the recent advent of state space models^{31,61–63} and linear recurrent networks⁶⁴ introduces a paradigm shift in sequence processing, where information is transported through constrained linear state transitions instead of being recurrently propagated between nonlinear neurons, as was previously the case in traditional recurrent neural networks⁶⁵. Similar to SNNs, state space models are obtained by discretizing continuous ordinary differential equations (ODEs) to form recurrent neural networks. Although spiking neuron models have always been derived from discretizing differential equations to obtain recurrent linear state transitions⁶⁶, earlier neuron models, such as the LIF neuron, lack the temporal dynamics necessary to effectively propagate time-sensitive information. The relation between SNNs and state space models has recently been discussed^{67,68}.

The adaptation discussed in this article directly acts on the dynamics of the neuron state. To keep the model simple and in order to study the effects of oscillatory dynamics in a clean manner, we did not include other recently proposed model extensions that can improve network performance. For example, oscillatory neuron dynamics can be combined with synaptic delays³⁸ or dendritic processing^{69,70}. Deckers et al.²⁴ reported promising results for a model that combines a variant of the adLIF model with synaptic delays. Further studies of such combinations, especially with the SE-adLIF model, constitute an interesting direction for future research.

In summary, we have shown that networks of adaptive LIF neurons provide a powerful model of computation for neuromorphic systems. Stability issues during training can provably be avoided by the use of a

suitable discretization method. Our results indicate that the properties of these neurons, in particular their sub-threshold oscillatory response, provide the basis for their spatio-temporal processing capabilities.

Methods

Details for simulations in Figure 2

For plots in Fig. 2b,c, we used the adLIF neuron parameters $\tau_w = 60$ ms, $\tau_u = 15$ ms, $a = 120$, $\vartheta = \infty$. For Fig. 2d-f the adLIF neuron parameters were $\tau_w = 200$ ms, $\tau_u = 125$ ms, $a = 100$, $\vartheta = \infty$. For Fig. 2a and g, parameters for the LIF neuron were $\tau_u = 125$ ms and $\vartheta = \infty$.

The spike trains for Fig. 2d and e were generated deterministically in the following way. We first computed a spike rate $s[t] \in [0, 0.2]$ for each time step t according to $s[t] = 0.2(0.5 + 0.5 \sin(2\pi t F \Delta t))$, where F is the oscillation frequency of the sinus signal in Hz ($F = 10$ Hz for Fig. 2d, $F = 7$ Hz for Fig. 2e), $\Delta t = 1$ ms the sampling time step, and $t \in [0, \dots, T]$. Then, the spike train $S[t]$ was computed by cumulatively summing over the spike rates in an integrate-and-fire manner with $v[t] = v[t-1] + s[t] - S[t]$, where $S[t] = \Theta(v[t] - 1)$ with Heaviside step function Θ and $v[0] = 0$.

Derivation of matrices $\bar{A}_{SE}$ and $\bar{B}_{SE}$ for the SE-adLIF neuron

Here, we provide the derivation of the matrices $\bar{A}_{SE}$ and $\bar{B}_{SE}$ for the SE-discretized adLIF neuron given in Eq. (11). To rewrite the state update equations of the SE-adLIF neuron model, given by

$$\hat{u}[k] = \alpha u[k-1] + (1-\alpha)(-w[k-1] + I[k]) \quad (15a)$$

$$w[k] = \beta w[k-1] + (1-\beta)(\alpha u[k] + bS[k]), \quad (15b)$$

into the canonical state-space representation

$$\mathbf{s}[k] = \begin{pmatrix} u[k] \\ w[k] \end{pmatrix} = \bar{A}_{SE} \mathbf{s}[k-1] + \bar{B}_{SE} \mathbf{x}[k], \quad (16)$$

we substitute $u[k]$ in Eq. (15b) by $\hat{u}[k]$ from Eq. (15a). Since we study the sub-threshold dynamics of the neuron (assuming $S[k] = 0$), we can substitute $\hat{u}[k]$ by $u[k]$, since $u[k] = \hat{u}[k] \cdot (1 - S[k])$. This yields the update equation for $w[k]$ given by

$$w[k] = \beta w[k-1] + (1-\beta) \left(\underbrace{a(\alpha u[k-1] + (1-\alpha)(-w[k-1] + I[k]))}_{\hat{u}[k]} + bS[k] \right). \quad (17)$$

Since this formulation gives the new state of adaptation variable $w[k]$ as function of the previous states $u[k-1]$ and $w[k-1]$, it can be transformed into a matrix formulation

$$\begin{pmatrix} u[k] \\ w[k] \end{pmatrix} = \underbrace{\begin{pmatrix} \alpha & -(1-\alpha) \\ a(1-\beta)\alpha & \beta - a(1-\beta)(1-\alpha) \end{pmatrix}}_{\bar{A}_{SE}} \begin{pmatrix} u[k-1] \\ w[k-1] \end{pmatrix} + \underbrace{\begin{pmatrix} (1-\alpha) & 0 \\ 0 & b(1-\beta) \end{pmatrix}}_{\bar{B}_{SE}} \begin{pmatrix} I[k] \\ S[k] \end{pmatrix}. \quad (18)$$

Proof of stability bounds for the continuous adLIF model

For all following analyses in Sections Proof of stability bounds for the continuous adLIF model to Stable ranges for intrinsic frequencies of

EF-adLIF, we consider the subthreshold regime, i.e., we assume a spike threshold $\vartheta = \infty$ such that $S[k] = 0$ for all k , and we assume no external inputs I .

In this section, we prove that the continuous-time adLIF neuron exhibits stable sub-threshold dynamics for all $a > -1$.

Lemma 1.2. The continuous adLIF neuron model from Eqs. (2) and (3) is stable in the sub-threshold regime for all $\tau_u > 0$, $\tau_w > 0$, $a > -1$.

Proof. In general, a continuous-time linear dynamical system $\dot{x} = Ax$ is Lyapunov-stable, if the real parts $\Re(\lambda_{1,2})$ of both eigenvalues $\lambda_{1,2}$ of matrix A satisfy $\Re(\lambda_{1,2}) \leq 0$ ⁷. For the adLIF model, the matrix A is given by

$$A = \begin{pmatrix} -\frac{1}{\tau_u} & -\frac{1}{\tau_u} \\ \frac{a}{\tau_w} & -\frac{1}{\tau_w} \end{pmatrix}, \quad (19)$$

with eigenvalues

$$\lambda_1 = \frac{-\tau_u - \tau_w + \sqrt{-4a\tau_u\tau_w + (\tau_u - \tau_w)^2}}{2\tau_u\tau_w} \quad (20)$$

$$\lambda_2 = \frac{-\tau_u - \tau_w - \sqrt{-4a\tau_u\tau_w + (\tau_u - \tau_w)^2}}{2\tau_u\tau_w}. \quad (21)$$

In the complex-valued case (where the discriminant $-4a\tau_u\tau_w + (\tau_u - \tau_w)^2 < 0$), the real part $\Re(\lambda_{1,2})$ is given by

$$\Re(\lambda_{1,2}) = \frac{-\tau_u - \tau_w}{2\tau_u\tau_w}, \quad (22)$$

where $\Re(\lambda_{1,2}) < 0$ since $\tau_u, \tau_w > 0$. In the case of real eigenvalues, $\lambda_2 < 0$ is always true (since Eq. (21) only contains negative terms), whereas $\lambda_1 \leq 0$ is only true if

$$\sqrt{-4a\tau_u\tau_w + (\tau_u - \tau_w)^2} \leq \tau_u + \tau_w \quad (23)$$

$$a \geq -\frac{(\tau_u + \tau_w)^2 - (\tau_u - \tau_w)^2}{4\tau_u\tau_w} \quad (24)$$

Hence, the continuous-time adLIF neuron is stable for all

$$a \geq -1. \quad (25)$$

This proves, that for $a \geq -1$, the continuous-time adLIF neuron is Lyapunov stable.

Derivation of stability bounds for EF-adLIF

In this section we derive the stability bounds of the EF-adLIF neuron with respect to parameter a . These bounds provide the basis for the subsequent proofs. The state update equations of the Euler-Forward discretized adLIF neuron are given by

$$\mathbf{s}(t) = \begin{pmatrix} \hat{u}(t) \\ \hat{w}(t) \end{pmatrix} = \bar{A}_{EF} \mathbf{s}(t) + \bar{B}_{EF} \mathbf{x}(t)$$

with

$$\bar{A}_{EF} = \begin{pmatrix} \alpha & -(1-\alpha) \\ a(1-\beta) & \beta \end{pmatrix} \quad \bar{B}_{EF} = \begin{pmatrix} (1-\alpha) & 0 \\ 0 & b(1-\beta) \end{pmatrix} \quad (26)$$

with $\alpha = e^{-\frac{\Delta t}{\tau_u}}$ and $\beta = e^{-\frac{\Delta t}{\tau_w}}$. This system is asymptotically stable, if the spectral radius ρ , given by $\rho = \max(|\lambda_1|, |\lambda_2|)$ is less than 1. We differentiate between two cases: complex-valued and real-valued eigenvalues. We assume given time constants $\tau_u, \tau_w > 0$ and calculate the bound as function of parameter a . To simplify the notation, we introduce $\bar{\alpha} = (1 - \alpha)$ and $\bar{\beta} = (1 - \beta)$.

Lemma 1.3. The EF-adLIF model is asymptotically stable and oscillating in the sub-threshold regime, for $a \in (a_0^{EF}, a_{\max}^{EF})$, with $a_0^{EF} = \frac{(\alpha - \beta)^2}{4(1 - \alpha)(1 - \beta)}$ and $a_{\max}^{EF} = \frac{1 - \alpha\bar{\beta}}{(1 - \alpha)(1 - \beta)}$. The real part of the eigenvalues is strictly positive and given by $\Re(\lambda_{1,2}) = \frac{\alpha + \beta}{2} > 0$.

Proof. The characteristic polynomial $\chi(\bar{A}_{EF})$ is given by

$$\chi(\bar{A}_{EF}) = \lambda^2 + \lambda(-\alpha - \beta) + \beta\alpha + a\bar{\alpha}\bar{\beta}, \quad (27)$$

with discriminant

$$\Delta_\lambda = (\alpha - \beta)^2 - 4a\bar{\alpha}\bar{\beta}. \quad (28)$$

$\chi(\bar{A}_{EF})$ admits complex solutions for $\Delta_\lambda < 0$ depending on a , with

$$a > \frac{(\alpha - \beta)^2}{4\bar{\alpha}\bar{\beta}} = a_0^{EF}, \quad (29)$$

Complex roots of $\chi(\bar{A}_{EF})$ yield the complex-conjugate eigenvalues

$$\lambda_{1,2} = \frac{\alpha + \beta}{2} \pm i \frac{\sqrt{-\Delta_\lambda}}{2}. \quad (30)$$

This proves that the complex eigenvalues have a strictly positive real part given by $\Re(\lambda_{1,2}) = \frac{\alpha + \beta}{2} > 0$. In that case the spectral radius ρ is defined as $\rho(\bar{A}_{EF}) = |\lambda_1| = |\lambda_2|$, where $|z| = \sqrt{\Re(z)^2 + \Im(z)^2}$ is the modulus of the complex number with

$$\rho(\bar{A}_{EF}) = \frac{1}{2} \sqrt{(\beta + \alpha)^2 - \Delta_\lambda} \quad (31)$$

$$= \frac{1}{2} \sqrt{(\beta + \alpha)^2 - (\beta - \alpha)^2 + 4a\bar{\alpha}\bar{\beta}} \quad (32)$$

$$= \sqrt{\alpha\beta + a\bar{\alpha}\bar{\beta}}. \quad (33)$$

In the complex regime, the system is stable when $\rho(\bar{A}_{EF}) < 1$, and thus,

$$a < \frac{1 - \alpha\bar{\beta}}{\bar{\alpha}\bar{\beta}} = a_{\max}^{EF}, \quad (34)$$

where $a_{\max}^{EF}$ gives the upper stability bound.

The system is asymptotically stable in the sub-threshold regime for $a \in (\frac{(\alpha - \beta)^2}{4\bar{\alpha}\bar{\beta}}, \frac{1 - \alpha\bar{\beta}}{\bar{\alpha}\bar{\beta}})$. In this bound, the system has complex eigenvalues and thus admits oscillations.

Lemma 1.4. EF-adLIF is asymptotically stable and not oscillating in the sub-threshold regime for $a \in (-1, a_0^{EF}]$ with $a_0^{EF} = \frac{(\alpha - \beta)^2}{4\bar{\alpha}\bar{\beta}}$.

Proof. The characteristic polynomial $\chi(\bar{A}_{EF})$ is given by

$$\chi(\bar{A}_{EF}) = \lambda^2 + \lambda(-\alpha - \beta) + \beta\alpha + a\bar{\alpha}\bar{\beta}, \quad (35)$$

with discriminant

$$\Delta_\lambda = (\alpha - \beta)^2 - 4a\bar{\alpha}\bar{\beta} \quad (36)$$

$\chi(\bar{A}_{EF})$ admits real solutions for $\Delta_\lambda \geq 0$ depending on a , with

$$\Delta_\lambda \geq 0 \quad (37)$$

$$(\alpha - \beta)^2 \geq 4a\bar{\alpha}\bar{\beta} \quad (38)$$

$$a \leq \frac{(\alpha - \beta)^2}{4\bar{\alpha}\bar{\beta}} = a_0^{EF}, \quad (39)$$

$\chi(\bar{A}_{EF})$ thus admits real solutions in the interval $(-\infty, a_0^{EF}]$ with real eigenvalues

$$\lambda_{1,2} = \frac{\alpha + \beta \pm \sqrt{\Delta_\lambda}}{2}. \quad (40)$$

For $a = a_0^{EF}$, the singular real root of $\chi(\bar{A}_{EF})$ is

$$\lambda_0 = \frac{\alpha + \beta}{2} < 1. \quad (41)$$

For lower values of a we have the following asymptotic behavior,

$$\lim_{a \rightarrow -\infty} \lambda_1 = \frac{\alpha + \beta + \sqrt{\Delta_\lambda}}{2} = \infty \quad (42)$$

$$\lim_{a \rightarrow -\infty} \lambda_2 = \frac{\alpha + \beta - \sqrt{\Delta_\lambda}}{2} = -\infty, \quad (43)$$

where λ_1 is the term with the higher absolute value and hence determines the spectral radius $\rho(\bar{A}_{EF})$. It exists a value $a_{\min}^{EF}$ such that $\lambda_1(a_{\min}^{EF}) = 1$,

$$\lambda_1 = \frac{\alpha + \beta + \sqrt{(\alpha - \beta)^2 - 4a\bar{\alpha}\bar{\beta}}}{2} = 1, \quad (44)$$

$$a_{\min}^{EF} = \frac{1 - \alpha - \beta + \alpha\bar{\beta}}{-\bar{\alpha}\bar{\beta}} = \frac{\bar{\alpha}\bar{\beta}}{-\bar{\alpha}\bar{\beta}} = -1, \quad (45)$$

and the system is unstable for $a < a_{\min}^{EF} = -1$. The system is thus asymptotically stable in the sub-threshold regime for $a \in (-1, a_0^{EF}]$. In this bound, the system has real eigenvalues and thus doesn't admit oscillations.

Corollary 1.4.1. In the sub-threshold regime (i.e. $\vartheta = \infty$), for $\tau_u, \tau_w, \Delta t > 0$, $\alpha = e^{-\frac{\Delta t}{\tau_u}}$ and $\beta = e^{-\frac{\Delta t}{\tau_w}}$, EF-adLIF is asymptotically stable for $a \in (-1, a_{\max}^{EF})$ with $a_{\max}^{EF} = \frac{1 - \alpha\bar{\beta}}{(1 - \alpha)(1 - \beta)}$.

This is a consequence of Lemma 1.3 and Lemma 1.4.

Derivation of the stability bounds of SE-adLIF

Analogously to Section Derivation of stability bounds for EF-adLIF, we can compute the stability bounds for the SE-discretized adLIF neuron (SE-adLIF) with respect to parameters τ_u, τ_w , and a . Again, we assume time constants $\tau_u, \tau_w > 0$ as given and calculate the stability bounds with respect to parameter a . Recall state transition

matrix $\bar{A}_{SE}$ from Eq. (11):

$$\bar{A}_{SE} = \begin{pmatrix} \alpha & -(1-\alpha) \\ \alpha(1-\beta)\alpha & \beta - \alpha(1-\beta)(1-\alpha) \end{pmatrix} \quad (46)$$

with $\alpha = e^{-\frac{\Delta t}{\tau_u}}$ and $\beta = e^{-\frac{\Delta t}{\tau_w}}$. To simplify notation, we again introduce $\bar{\alpha} = (1-\alpha)$ and $\bar{\beta} = (1-\beta)$.

Lemma 1.5. SE-adLIF is stable and oscillating in the sub-threshold regime for $a \in (a_1^{SE}, a_2^{SE})$, with $a_1^{SE} = \frac{(\sqrt{\bar{\beta}} - \sqrt{\bar{\alpha}})^2}{\bar{\beta}\bar{\alpha}}$ and $a_2^{SE} = \frac{(\sqrt{\bar{\beta}} + \sqrt{\bar{\alpha}})^2}{\bar{\beta}\bar{\alpha}}$. The spectral radius is independent of a and given by $\rho = \sqrt{\bar{\beta}\bar{\alpha}} < 1$.

Proof. The characteristic polynomial of $\chi(\bar{A}_{SE})$ is

$$\chi(\bar{A}_{SE}) = \lambda^2 + \lambda(\bar{\beta}\bar{\alpha}a - \beta - \alpha) + \beta\alpha, \quad (47)$$

with discriminant Δ_λ given by

$$\Delta_\lambda = (\bar{\beta}\bar{\alpha}a - \beta - \alpha)^2 - 4\beta\alpha \\ = \bar{\beta}\bar{\alpha} \left[\bar{\beta}\bar{\alpha}a^2 - 2(\beta + \alpha)a + \frac{(\beta - \alpha)^2}{\bar{\beta}\bar{\alpha}} \right]. \quad (48)$$

$\chi(\bar{A}_{SE})$ gives complex solutions for $\Delta_\lambda < 0$, which depends on the range of negative values of the polynomial part $p(a)$ of Δ_λ , given by

$$p(a) = \bar{\beta}\bar{\alpha}a^2 - 2(\beta + \alpha)a + \frac{(\beta - \alpha)^2}{\bar{\beta}\bar{\alpha}}. \quad (49)$$

From this polynomial, we can again compute a discriminant Δ_a as

$$\Delta_a = 16\beta\alpha. \quad (50)$$

Since $\Delta_a > 0$ the roots of this polynomial are given by two values a_1^{SE} and a_2^{SE} according to

$$a_1^{SE} = \frac{(\sqrt{\bar{\beta}} - \sqrt{\bar{\alpha}})^2}{\bar{\beta}\bar{\alpha}}, \quad a_2^{SE} = \frac{(\sqrt{\bar{\beta}} + \sqrt{\bar{\alpha}})^2}{\bar{\beta}\bar{\alpha}}. \quad (51)$$

As the coefficient $\bar{\beta}\bar{\alpha}$ in Eq. (48) is always positive, Δ_λ is negative when $a_1^{SE} < a < a_2^{SE}$ resulting in complex-conjugate eigenvalues $\lambda_{1,2}$, given by the roots of $\chi(\bar{A}_{SE})$, implying oscillatory behavior of the membrane potential. These eigenvalues are given by

$$\lambda_{1,2} = -\frac{(\bar{\beta}\bar{\alpha}a - \beta - \alpha)}{2} \pm i\frac{\sqrt{-\Delta_\lambda}}{2}. \quad (52)$$

In that case the spectral radius is defined as $\rho(A) = |\lambda_1| = |\lambda_2|$, where $|z| = \sqrt{\text{Re}(z)^2 + \text{Im}(z)^2}$ is the modulus of the complex eigenvalues, such that

$$r = \rho(A) = \frac{1}{2} \sqrt{(\bar{\beta}\bar{\alpha}a - \beta - \alpha)^2 - \Delta_\lambda} \quad (53)$$

$$= \frac{1}{2} \sqrt{(\bar{\beta}\bar{\alpha}a - \beta - \alpha)^2 - (\bar{\beta}\bar{\alpha}a - \beta - \alpha)^2 + 4\beta\alpha} \quad (54)$$

$$= \sqrt{\beta\alpha}. \quad (55)$$

Hence, the spectral radius, which we also refer to as the decay rate r in the main text, is $\sqrt{\beta\alpha}$ which is always strictly less than 1 due to

$\beta, \alpha \in (0, 1)$. Therefore, the SE-adLIF neuron is stable over the entire range of parameters (provided $\tau_u, \tau_w > 0$) where the matrix $\bar{A}_{SE}$ exhibits complex eigenvalues.

Lemma 1.6. SE-adLIF is stable and not oscillating in the sub-threshold regime for $a \in (-1, a_1^{SE}] \cup [a_2^{SE}, a_{\max}^{SE})$ with $a_{\max}^{SE} = \frac{(1+\beta)(1+\alpha)}{(1-\beta)(1-\alpha)}$ and a_1^{SE} , a_2^{SE} as defined in Lemma 1.5.

Proof. The characteristic polynomial of $\chi(\bar{A}_{SE})$ is

$$\chi(\bar{A}_{SE}) = \lambda^2 + \lambda(\bar{\beta}\bar{\alpha}a - \beta - \alpha) + \beta\alpha, \quad (56)$$

with discriminant Δ_λ given by

$$\Delta_\lambda = (\bar{\beta}\bar{\alpha}a - \beta - \alpha)^2 - 4\beta\alpha \\ = \bar{\beta}\bar{\alpha} \left[\bar{\beta}\bar{\alpha}a^2 - 2(\beta + \alpha)a + \frac{(\beta - \alpha)^2}{\bar{\beta}\bar{\alpha}} \right]. \quad (57)$$

The derivations from Lemma 1.5 imply that $\chi(\bar{A}_{SE})$ admits real solutions in the intervals $(-\infty, a_1^{SE}]$ and $[a_2^{SE}, \infty)$, yielding real-valued eigenvalues,

$$\lambda_{1,2} = \frac{-(\bar{\beta}\bar{\alpha}a - \beta - \alpha) \pm \sqrt{\Delta_\lambda}}{2}. \quad (58)$$

For $a = a_1^{SE}$ and $a = a_2^{SE}$, $\Delta_\lambda = 0$ and we have two singular solutions,

$$\lambda_1^0 = -\frac{(\bar{\beta}\bar{\alpha}a_1^{SE} - \beta - \alpha)}{2} = \sqrt{\beta\alpha} \quad (59)$$

$$\lambda_2^0 = -\frac{(\bar{\beta}\bar{\alpha}a_2^{SE} - \beta - \alpha)}{2} = -\sqrt{\beta\alpha}. \quad (60)$$

For both singular solutions the spectral radius is $\rho(\bar{A}_{SE}) = |r_1^0| = |r_2^0| = \sqrt{\beta\alpha}$ which is less than one, resulting in asymptotic stability.

In order to study the stability for $a < a_1^{SE}$ and $a > a_2^{SE}$ we need to determine the asymptotic behavior of $\lambda_{1,2}$ for $a \rightarrow -\infty$ and $a \rightarrow \infty$, as it allows us to determine which eigenvalue will constrain the stability of the system. In the following propositions, we prove the stability of the system in the intervals $(-1, a_1^{SE}]$ and $[a_2^{SE}, a_{\max}^{SE})$ independently.

Proposition 1. SE-adLIF is asymptotically stable for $a \in (-1, a_1^{SE}]$. For $a < a_1^{SE}$, $\rho(\bar{A}_{SE})$ is determined by $|\lambda_1|$ as for $a \rightarrow -\infty$, we have

$$\lim_{a \rightarrow -\infty} \lambda_1 = \lim_{a \rightarrow -\infty} \frac{-(\bar{\beta}\bar{\alpha}a - \beta - \alpha) + \sqrt{(\bar{\beta}\bar{\alpha}a - \beta - \alpha)^2 - 4\beta\alpha}}{2} = \infty \quad (61)$$

$$\lim_{a \rightarrow -\infty} \lambda_2 = \lim_{a \rightarrow -\infty} \frac{-(\bar{\beta}\bar{\alpha}a - \beta - \alpha) - \sqrt{(\bar{\beta}\bar{\alpha}a - \beta - \alpha)^2 - 4\beta\alpha}}{2} = 0. \quad (62)$$

Thus $\lambda_1^0 < 1 < \lim_{a \rightarrow -\infty} \lambda_1$. We can find a value $a_{\min}^{SE}$ such that $\lambda_1(a_{\min}^{SE}) = 1$, given by

$$\lambda_1 = \frac{-(\bar{\beta}\bar{\alpha}a_{\min}^{SE} - \beta - \alpha) + \sqrt{(\bar{\beta}\bar{\alpha}a_{\min}^{SE} - \beta - \alpha)^2 - 4\beta\alpha}}{2} = 1 \quad (63)$$

$$a_{\min}^{SE} = \frac{-\beta\alpha + \beta + \alpha - 1}{\bar{\beta}\bar{\alpha}} = \frac{-\bar{\beta}\bar{\alpha}}{\bar{\beta}\bar{\alpha}} = -1. \quad (64)$$

Proposition 2. SE-adLIF is asymptotically stable for $a \in [a_2^{\text{SE}}, a_{\text{max}}^{\text{SE}}]$ with $a_{\text{max}}^{\text{SE}} = \frac{(1+\beta)(1+\alpha)}{(1-\beta)(1-\alpha)}$.

For $a > a_2^{\text{SE}}$, $\rho(\bar{A}_{\text{SE}})$ is determined by $|\lambda_2|$ as for $a \rightarrow \infty$, we have

$$\lim_{a \rightarrow \infty} \lambda_1 = \lim_{a \rightarrow \infty} \frac{-(\bar{\beta}\alpha a - \beta - \alpha) + \sqrt{(\bar{\beta}\alpha a - \beta - \alpha)^2 - 4\beta\alpha}}{2} = 0 \quad (65)$$

$$\lim_{a \rightarrow \infty} \lambda_2 = \lim_{a \rightarrow \infty} \frac{-(\bar{\beta}\alpha a - \beta - \alpha) - \sqrt{(\bar{\beta}\alpha a - \beta - \alpha)^2 - 4\beta\alpha}}{2} = -\infty. \quad (66)$$

Thus $\lim_{a \rightarrow \infty} \lambda_2 < -1 < \lambda_2^0$. We can find a value $a_{\text{max}}^{\text{SE}}$ such that $\lambda_2(a_{\text{max}}^{\text{SE}}) = -1$, given by

$$\lambda_2 = \frac{-(\bar{\beta}\alpha a_{\text{max}}^{\text{SE}} - \beta - \alpha) - \sqrt{(\bar{\beta}\alpha a_{\text{max}}^{\text{SE}} - \beta - \alpha)^2 - 4\beta\alpha}}{2} = -1 \quad (67)$$

$$a_{\text{max}}^{\text{SE}} = \frac{1 + \beta\alpha + \beta + \alpha}{\bar{\beta}\alpha} = \frac{(1 + \alpha)(1 + \beta)}{\bar{\beta}\alpha} \quad (68)$$

For all, $a > a_{\text{max}}^{\text{SE}}$, it follows that $\rho(\bar{A}_{\text{SE}}) = |\lambda_2| > 1$ and the system is unstable.

Corollary 1.6.1. In the sub-threshold regime (i.e. $\vartheta = \infty$), for $\tau_u, \tau_w, \Delta t > 0$ and $\alpha = e^{-\frac{\Delta t}{\tau_u}}$ and $\beta = e^{-\frac{\Delta t}{\tau_w}}$, the Symplectic-Euler discretized adLIF neuron (SE-adLIF) is asymptotically stable for all $a \in (-1, a_{\text{max}}^{\text{SE}})$ with $a_{\text{max}}^{\text{SE}} = \frac{(1+\beta)(1+\alpha)}{(1-\beta)(1-\alpha)}$.

This is a consequence of Lemma 1.5 and Lemma 1.6.

Proof of Theorem 1.1

Theorem 1.1 states that for each choice of $\tau_u > 0$, $\tau_w > 0$, and each intrinsic frequency $f \in [0, f_N]$, there is a unique parameter value $[a_1^{\text{SE}}, a_2^{\text{SE}}]$ such that the SE-adLIF neuron with these parameters has intrinsic frequency f and vice versa, while being asymptotically stable in the sub-threshold regime. In other words, the neuron model can exhibit the full range of intrinsic frequencies for any setting of $\tau_u > 0$, $\tau_w > 0$ in a stable manner. In the following we proof Theorem 1.1.

Proof. We first show that for an arbitrary intrinsic frequency $f \in ([0, f_N])$, there exists a value a such that an SE-adLIF neuron with arbitrary $\tau_u > 0$ and $\tau_w > 0$ oscillates with f . To show this, we consider an SE-adLIF neuron in the sub-threshold regime with an arbitrary choice of $\tau_u > 0$ and $\tau_w > 0$. Let $g_{\tau_u, \tau_w}: [a_1^{\text{SE}}, a_2^{\text{SE}}] \rightarrow [0, f_N]$ be the function that maps parameter a of the neuron to the neuron's intrinsic frequency f . We prove in the following that g_{τ_u, τ_w} is a bijection.

As proven in the Lemma 1.5, in the range $a \in (a_1^{\text{SE}}, a_2^{\text{SE}})$, with $a_1^{\text{SE}} = \frac{(\sqrt{\beta} - \sqrt{\alpha})^2}{\bar{\beta}\alpha}$ and $a_2^{\text{SE}} = \frac{(\sqrt{\beta} + \sqrt{\alpha})^2}{\bar{\beta}\alpha}$, $\bar{A}_{\text{SE}}$ has complex eigenvalues $\lambda_{1,2}$ given by Eq. (52).

Let $a \in [a_1^{\text{SE}}, a_2^{\text{SE}}]$ for an arbitrary choice of $\tau_u > 0$ and $\tau_w > 0$, and $\lambda_1(a)$ be the complex eigenvalue of $\bar{A}_{\text{SE}}$ as function of a with positive argument $\arg(\lambda_1) \geq 0$.

We claim there is a natural bijection $\phi(a)$ between $[a_1^{\text{SE}}, a_2^{\text{SE}}]$ and $[0, \pi]$. We first show that $\cos(\phi(a))$ is a bijection between $[a_1^{\text{SE}}, a_2^{\text{SE}}]$ and $[-1, 1]$. The bijection to $[0, \pi]$ follows directly from $\arccos(x): [-1, 1] \rightarrow [0, \pi]$ defined as a bijective function on its principal values $[0, \pi]$.

We have the trigonometric relation $\Re(\lambda_1(a)) = r \cos(\phi(a))$ with $r = |\lambda_1(a)| = \sqrt{\beta\alpha}$. $\cos(\phi(a)) = \frac{\Re(\lambda_1(a))}{\sqrt{\beta\alpha}}$ is a bijection between $[a_1^{\text{SE}}, a_2^{\text{SE}}]$ and $[-1, 1]$. $\cos(\phi(a))$ is surjective, since it is continuous in a ,

$\cos(\phi(a_1^{\text{SE}})) = 1$, and $\cos(\phi(a_2^{\text{SE}})) = -1$. $\cos(\phi(a))$ is injective, as for $a \in (a_1^{\text{SE}}, a_2^{\text{SE}})$, $\Re(\lambda_1(a))$ is a strictly decreasing continuous function, which follows from the fact that its derivative $\Re'(\lambda_1(a)) = -\frac{\bar{\beta}\alpha}{2} < 0$ for all $a \in (a_1^{\text{SE}}, a_2^{\text{SE}})$, and $r = \sqrt{\beta\alpha}$ is a positive constant.

It follows that $\phi(a) = \arccos \frac{\Re(\lambda_1(a))}{\sqrt{\beta\alpha}}$ defines a bijective function between $[a_1^{\text{SE}}, a_2^{\text{SE}}]$ and $[0, \pi]$, with $\phi(a_1^{\text{SE}}) = \arccos(1) = 0$ and $\phi(a_2^{\text{SE}}) = \arccos(-1) = \pi$.

Since the frequency in Hertz is defined by $f = \frac{\phi}{2\pi\Delta t}$, we have shown that

$$g_{\tau_u, \tau_w}: [a_1^{\text{SE}}, a_2^{\text{SE}}] \rightarrow [0, f_N] \quad (69)$$

$$a \mapsto \frac{\phi(a)}{2\pi\Delta t}$$

is a bijective function with $g_{\tau_u, \tau_w}(a_1^{\text{SE}}) = 0$ and $g_{\tau_u, \tau_w}(a_2^{\text{SE}}) = f_N$, the Nyquist frequency.

Hence, we have shown that for an arbitrary intrinsic frequency $f \in ([0, f_N])$, there exists a value a such that an SE-adLIF neuron with arbitrary $\tau_u > 0$ and $\tau_w > 0$ oscillates with f .

Second, we have to show that an SE-adLIF neuron with arbitrary parameters (a , $\tau_u > 0$, $\tau_w > 0$) and intrinsic oscillation frequency $f \in ([0, f_N])$ is asymptotically stable with decay rate $r = \sqrt{\alpha\beta} < 1$ where $\alpha = e^{-\frac{\Delta t}{\tau_u}}$ and $\beta = e^{-\frac{\Delta t}{\tau_w}}$. Above we have shown that for parameter $a \in [a_1^{\text{SE}}, a_2^{\text{SE}}]$ a bijective mapping to each frequency $f \in ([0, f_N])$ exists. The asymptotic stability follows from this fact in combination with Lemma 1.5.

Stable ranges for intrinsic frequencies of EF-adLIF

We consider the parameterizations of EF-adLIF and SE-adLIF neurons where the discretized transition matrix $\bar{A}_{\text{SE}}$ has complex eigenvalues. In that case, the neuron exhibits oscillations of intrinsic frequency f determined by the angle $\phi = \arg(\lambda_1)$ of the complex eigenvalue λ_1 , see also Sections Derivation of stability bounds for EF-adLIF and Derivation of the stability bounds of SE-adLIF. This angle determines the angle of 'rotation' of the state for each time step, from which we can infer the intrinsic frequency by $f = \frac{\phi}{2\pi} f_s$ where $f_s = \frac{1}{\Delta t}$ is the sampling frequency. We define the Nyquist frequency as half the sampling frequency $f_N = \frac{f_s}{2}$.

Lemma 1.7. EF-adLIF neurons can oscillate with intrinsic frequencies f bounded by $f \in [0, \frac{f_N}{2})$

Proof. We proved in Lemma 1.3 that in the range $a \in (a_0^{\text{EF}}, a_{\text{max}}^{\text{EF}})$, with $a_0^{\text{EF}} = \frac{(\alpha-\beta)^2}{4\alpha\beta}$ and $a_{\text{max}}^{\text{EF}} = \frac{1-\alpha\beta}{\alpha\beta}$, the EF-adLIF neuron is asymptotically stable and has complex eigenvalues $\lambda_{1,2}$ given by Eq. (30).

λ_1 is restricted to the right half-plane of the complex plane, which follows directly from the fact that, by definition $\Im(\lambda_1) > 0$ and $\Re(\lambda_1) = \frac{\beta + \alpha}{2} > 0$, for all $\tau_u > 0$ and $\tau_w > 0$. Hence, the argument is restricted to $0 \leq \phi < \frac{\pi}{2}$. Since the frequency in Hertz is defined by $f = \frac{\phi}{2\pi\Delta t}$, this results in an upper bound on the oscillation frequency of $f < \frac{f_N}{2}$ with f_N the Nyquist frequency.

Note, that this upper bound is approached in the limit $\tau_u, \tau_w \rightarrow 0$, but the maximum frequency is much lower for realistic values of τ_u and τ_w , as shown in Fig. 3f.

Lemma 1.8. For EF-adLIF, as τ_u and τ_w increase, the stable frequency bandwidth of EF-adLIF asymptotically converges towards 0.

Proof. Recall that $\alpha = e^{-\frac{\Delta t}{\tau_u}}$ and $\beta = e^{-\frac{\Delta t}{\tau_w}}$ for $\tau_u > 0$ and $\tau_w > 0$. We evaluate $\Im(\lambda_1)$ at the stability boundary $a_{\text{max}}^{\text{EF}}$ and obtain

Table 4 | Task- and model-specific hyperparameters

		lr	# neurons	# layers	q	α (SLAYER)	c (SLAYER)	Ep.	$[\tau_u^{\min}, \tau_u^{\max}]$	$[\tau_w^{\min}, \tau_w^{\max}]$	dropout	τ_{out}	batch size
SHD	SE	0.01	128/360	1/2	120	5	0.4	300	[5, 25]	[60, 300]	15%	15	256
	EF	0.01	360	2	60	5	0.4	300	[5, 25]	[60, 300]	15%	15	256
	LIF	0.01	360	2	–	5	0.1	300	[5, 150]	–	15%	15	256
SSC	SE	0.006	720	2	120	5	0.4	40	[5, 25]	[60, 300]	15%	15	256
	EF	0.006	720	2	60	5	0.4	40	[5, 25]	[60, 300]	15%	15	256
	LIF	0.006	720	2	–	5	0.1	40	[5, 150]	–	15%	15	256
ECG	SE	0.01	36	1/2	120	5	0.2	400	[5, 25]	[60, 300]	15%	3	64
	EF	0.01	36	1	60	5	0.2	400	[5, 25]	[60, 300]	15%	3	64
	LIF	0.01	36	1	–	5	0.1	400	[5, 150]	–	15%	3	64
BSD	SE	0.01	512	1	120	5	0.4	400	[5, 25]	[60, 300]	0%	15	128
	LIF	0.006	510	1	–	5	0.2	400	[5, 50]	–	0%	15	128
spring-mass	SE/EF	0.01	[25, 3200]	1	228/65	5	0.4	200	[5, 25]	[60, 300]	0%	[1, 20]	256
	LIF	0.01	[27, 3202]	1	–	10	0.5	200	[1, 25]	–	0%	[1, 20]	256
	LSTM	0.001	[13, 1600]	1	–	–	–	200	–	–	0%	[1, 20]	256
audio comp.	SE/EF	$5 \cdot 10^{-4}$	300	4	120/20	5	0.4	10	[5, 25]	[30, 300]	0%	[1, 10]	128
	LIF	$5 \cdot 10^{-4}$	302	4	–	5	0.1	10	[5, 100]	–	0%	[1, 10]	128

Our experiments use a spike threshold of $\theta = 1$ and an integration time-step $\Delta t = 1$ ms, except for spring-mass ($\Delta t = 2.5$ ms) and audio comp. (trainable threshold).

lr: learning rate for ADAM optimizer, Ep.: number of training epochs, τ_{out} : Membrane time constants of output layer (leaky integrator), dropout: dropout rate, q: coefficient of the reparameterization of a and b (see Eq. (75) and (76)).

$\Re(\lambda_1(a^{\text{EF}}_{\text{max}})) = \frac{\sqrt{-(\alpha+\beta)^2+4}}{2}$. At this point the modulus is $r = 1$ and the maximum radial frequency is thus,

$$\phi_{\text{max}} = \arcsin \frac{\sqrt{-(\alpha+\beta)^2+4}}{2}. \quad (70)$$

As $\lim_{\tau \rightarrow \infty} e^{-\frac{\Delta t}{\tau}} = 1$, it is clear that $\lim_{\tau_u, \tau_w \rightarrow \infty} \phi_{\text{max}} = 0$.

Benchmark Datasets and Preprocessing for Tables 1 and 2

The SHD dataset was preprocessed by sum-pooling spikes temporally using bins of 4 ms (as in¹⁷) and spatially using a bin size of 5 channels, such that its input dimension was reduced from 700 to 140 channels, as in³⁸. Note, that any resulting preprocessed sample $X \in \mathbb{N}_+^{T \times 140}$ of length T thereby has integer-valued entries, where each entry x_{kj} denotes the number of spikes occurring during the k -th 4 ms time window within the j -th group of 5 channels in the raw data. We padded samples that were shorter than 250 time steps to a minimum length of 250 with zeros to ensure that the network has enough ‘time’ for a decision, but kept longer sequences as they were. We applied the same temporal and spatial pooling to the SSC dataset, but zero-padded the samples to a minimum length of 300 time steps, since the relevant part of the data usually appears later in the sequence in SSC.

For the ECG dataset, we used the preprocessed files from²⁵, where the two-channel ECG signals from the QT database⁷² were pre-processed using a level-crossing encoding. For details refer to²⁵, Methods section. We considered two cases of the SHD dataset, one where we validated on the test set and chose the best epoch based on this validation (=test) accuracy, and one where we used 20% of the training set as held-out validation set and performed testing on the test set, using the weights of the epoch with the highest validation accuracy. For SSC, a distinct validation set was provided. For ECG, we used a fraction of 5% of samples from the training set for validation.

Training and Hyperparameter Search Details for all Tasks

A summary of general hyperparameters used in our experiments is shown in Table 4.

Optimizer and surrogate gradient. We trained the SNNs using the SLAYER surrogate gradient⁸ defined by $\frac{\partial S}{\partial v} = \frac{c\alpha}{2 \exp(\alpha|v|)}$ with α and c according to Table 4 for all experiments. We found that careful choice of the scale parameter c is crucial to achieve good performance for all networks in which the SLAYER gradient is used. A too high c results in an exploding gradient, whereas a too small c can result in vanishing gradients. We trained all networks with back-propagation through time with minibatches using PyTorch²⁸. We used the ADAM⁷³ optimization algorithm for all experiments with $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. For the LIF and adLIF models we detached the spike from the gradient during the reset, such that $u[k] = \hat{u}[k] \cdot (1 - \text{sg}(S[k]))$ where sg is the stop-gradient function with $\text{sg}(x) = x$ and $\frac{\partial}{\partial x} \text{sg}(x) = 0$. We applied gradient clipping and rescaled the gradient if it exceeded a norm of 10 (audio compression task) or 1.5 (all other tasks).

Network Output Layer. The output layer of the network consisted of leaky integrator (LI) neurons of the same number as classes in the task at hand. The LI membrane potential at time step k is given by

$$u[k] = \gamma u[k-1] + (1 - \gamma)I[k] \quad (71)$$

with $\gamma = \exp\left(-\frac{\Delta t}{\tau_{\text{out}}}\right)$, time constant τ_{out} and input current $I[k]$. LI neurons don’t emit spikes, hence they lack a threshold and reset mechanism, instead their output is their membrane potential $u[k]$.

Loss functions. In all tasks, the last layer of the network consisted of leaky integrator neurons that match the number of classes of the corresponding task, or the number of masses in case of the oscillatory dynamical system trajectory prediction task. For the audio reconstruction task we quantized the waveform into discrete bins and used a multi-component loss (please refer to Section Details for the audio compression task). For the SHD and BSD tasks, the loss function was given by $L = \text{CrossEntropy}(\sum_t \text{softmax}(\mathbf{y}_t), \mathbf{c})$ for one-hot encoded target class $\mathbf{c}$ and network output $\mathbf{y}$ at time step t . We discarded the network output of the first 10 time steps for the calculation of the loss for SHD, whereas for the BSD task we discarded the output of the first 80% of time steps. For SSC, we used the loss function

$L = \text{CrossEntropy}(\text{softmax}(\sum_t \mathbf{y}_t), \mathbf{c})$. Again, we discarded the network output of the first 10 time steps.

For the ECG dataset, the loss was computed on a per-time-step level as $L = \sum_t \text{CrossEntropy}(\text{softmax}(\mathbf{y}_t), \mathbf{c}_t)$ where $\mathbf{c}_t$ is the label per time step t . For the trajectory prediction task, we used the mean-squared-error (MSE) loss over the temporal sequence, $L = \frac{1}{n \times T} \sum_{t=1}^T \sum_{j=1}^n (\mathbf{y}_j^* - \mathbf{y}_j^t)^2$ where T is the number of time steps in the sequence, n the number of masses and $\mathbf{y}^*$ is the ground truth of the masses' displacements.

Hyperparameter tuning. The hyperparameters for the adLIF network were tuned using a mixture of manual tuning and the Hyperband algorithm⁷⁴, mainly on the SHD classification task. Since the search space of hyperparameters of the adLIF model is quite large, we neither ran exhaustive searches on the ranges for time constants τ_u and τ_w , nor the ranges of parameters a and b (for details on how we train these parameters see next section). We selected the ranges based on our stability analyses to ensure stable neurons, and on previous empirical results on similar models¹⁶. For SSC and ECG we manually tuned only the number of neurons, the learning rate and the SLAYER gradient scale c and kept the values for other hyperparameters same as for the SHD task. We found, that the out-of-the-box performance for these hyperparameters was already very competitive, providing a solid choice as a starting point. For the trajectory prediction task, we performed the hyperparameter search using the Hyperband algorithm⁷⁴. Only the LIF model was highly sensitive to its hyperparameters—we found that the time constants τ_u and the hyperparameters of the SLAYER gradient function were critical for learning. For the LSTM network, we found that the learning rate should be lower than for LIF and adLIF, and similar to⁷⁵, we found that for the model to converge, the forget gate of LSTM should be initially biased to one. For EF-adLIF and SE-adLIF, the performances were mostly invariant to hyperparameter changes, so we conserved the hyperparameters found for SHD and only restricted the parameters a to correspond to the frequency bandwidths described in Accurate prediction of dynamical system trajectories.

Reparameterization and initialization. For all models, we trained the time constants τ_u and τ_w (except for LIF, where no τ_w occurs) in addition to the synaptic weights. For all tasks except the audio compression task we reparameterized the time constants via

$$\tau_x = \tau_x^{\min} + \theta_x (\tau_x^{\max} - \tau_x^{\min}), \quad (72)$$

with $x \in \{u, w\}$, trained parameters θ_x clipped to the interval $\theta_x \in [0, 1]$ during training and $\tau_x^{\min}$ and $\tau_x^{\max}$ as hyperparameters according to Table 4. We found this reparameterization useful if the ADAM optimizer is used, since its dynamic adjustment of the learning rates expects all parameters to be roughly in the same order of magnitude, which is usually not the case for joint training of neuron time constants (order of 10^1 to 10^2) and synaptic weights (order of 10^0). In the audio compression task, we found it beneficial to use a different reparameterization scheme, where we directly parameterized α and β according to:

$$\alpha = \sigma(\theta_\alpha) \exp\left(-\frac{\Delta t}{\tau_u^{\min}}\right) + (1 - \sigma(\theta_\alpha)) \exp\left(-\frac{\Delta t}{\tau_u^{\max}}\right), \quad (73)$$

$$\beta = \sigma(\theta_\beta) \exp\left(-\frac{\Delta t}{\tau_w^{\min}}\right) + (1 - \sigma(\theta_\beta)) \exp\left(-\frac{\Delta t}{\tau_w^{\max}}\right), \quad (74)$$

with parameters θ_α and θ_β and logistic sigmoid function σ .

For the parameters a and b of the adLIF model we applied a reparameterization similar to Eq. (72), where

$$\begin{aligned} a &= q\hat{a} \\ b &= qb, \end{aligned} \quad (75) \quad (76)$$

with hyperparameter q and trained parameters $\hat{a}$ and $\hat{b}$, clipped to $\hat{a} \in [0, 1]$ and $\hat{b} \in [0, 2]$ during training, except for the audio compression task (refer to Section Details for the audio compression task). We restrict the range of parameters to $a, b > 0$ to avoid instabilities caused by a positive feed-back loop between adaptation variable w and membrane potential u . This constraint has also been discussed in²⁴. We initialized the feed-forward weights in all models uniformly in the interval $[-\sqrt{\frac{1}{\text{fan}_{\text{in}}}}, \sqrt{\frac{1}{\text{fan}_{\text{in}}}}]$ with fan_{in} as the number of inbound synaptic feed-forward connections, and all recurrent weights according to the orthogonal method described in⁷⁶ with a gain factor of 1. $\hat{a}, \hat{b}, \theta_u$ and θ_w are initialized uniformly over their respective range, as stated above. Note, that these are neuron-level parameters, i.e. each neuron has individual values of these parameters.

Details for the dynamical system trajectory prediction task

As illustrated in Fig. 4a, we consider a system of n masses connected with $n + 1$ springs where each mass (except the two outermost) is connected to two other masses by a spring. The rightmost and leftmost masses are connected to one mass and the fixed support each (see schematic in Fig. 4a). The temporal evolution of the displacements of the masses $\mathbf{x}(t) \in \mathbb{R}^n$ can be written using the equation of motion

$$M\ddot{\mathbf{x}}(t) + S\mathbf{x}(t) = \mathbf{0}, \quad (77)$$

where $M \in \mathbb{R}^{n \times n}$ is a diagonal matrix with diagonal entries corresponding to the masses in kg, while $S \in \mathbb{R}^{n \times n}$ corresponds to the matrix of interaction between the masses determined by the spring coefficients. S is given by

$$S_{ij} = \begin{cases} s_i + s_{i+1} & \text{if } i=j \\ -s_i & \text{if } j=i-1 \\ -s_{i+1} & \text{if } j=i+1 \\ 0 & \text{otherwise} \end{cases} \quad (78)$$

with $\mathbf{s} \in \mathbb{R}^{n+1}$ as the spring coefficients in N m⁻¹. We solve this system by considering velocity vector $\mathbf{v}(t) = \dot{\mathbf{x}}(t)$, which results in an equivalent system of $2n$ equations of the form

$$\underbrace{\begin{pmatrix} I & \mathbf{0} \\ \mathbf{0} & M \end{pmatrix}}_A \begin{pmatrix} \dot{\mathbf{x}}(t) \\ \mathbf{v}(t) \end{pmatrix} = \underbrace{\begin{pmatrix} \mathbf{0} & I \\ -S & \mathbf{0} \end{pmatrix}}_B \begin{pmatrix} \mathbf{x}(t) \\ \mathbf{v}(t) \end{pmatrix}, \quad (79)$$

with $\mathbf{0}$ and $I \in \mathbb{R}^{n \times n}$ representing the zero-valued and the identity matrices respectively. The system has a homogeneous solution of the form

$$\begin{pmatrix} \mathbf{x}(t) \\ \mathbf{v}(t) \end{pmatrix} = \exp(A^{-1}Bt) \begin{pmatrix} \mathbf{x}(0) \\ \mathbf{v}(0) \end{pmatrix}, \quad (80)$$

where $\mathbf{x}(0), \mathbf{v}(0) \in \mathbb{R}^n$ correspond to the initial conditions of displacements and velocities of the masses respectively.

For each independent trial in Fig. 4e, we constructed an individual spring-mass system with $n = 4$ masses and 5 springs by generating random spring coefficients s_i for each spring i from a uniform distribution over the interval [500, 10000] N m⁻¹, whereas the masses' magnitudes were fixed to 1 kg. The system's parameters were then held

fixed throughout the trial. We sampled 4096 trajectories from this system with random initial conditions to construct the dataset for a trial. Each network was then trained on this per-trial dataset for 200 epochs. We repeated the experiment for 5 independent trials.

The chosen range of spring coefficients resulted in eigenfrequencies of the systems between approximately 2 to 32 Hz.

For Fig. 4g, we trained our models on different systems with increasing minimal and maximal frequency. Since the frequency of oscillations is determined by S and the spring coefficients, we seek to associate a range of frequencies with a range of spring coefficients from which to sample system parameters. The theoretical frequency bandwidth associated with Eq. (77) can be approximated by calculating the eigenfrequency of the system when all springs are set to the same coefficient s , resulting in the matrix $\hat{S}$ of spring coefficients

$$\hat{S}_{ij} = \begin{cases} 2s & \text{if } i=j \\ -s & \text{if } j=i-1 \\ -s & \text{if } j=i+1 \\ 0 & \text{otherwise} \end{cases} \quad (81)$$

In the case of unit masses, we can formulate an eigenvalue problem from the system defined by Eq. (77) as

$$\hat{S}v_j = \omega_j^2 v_j, \quad (82)$$

where v_j is the eigenvector associated to the eigenvalue ω_j^2 and ω_j is the corresponding radial frequency. $\hat{S}$ is a tridiagonal Toeplitz matrix, as such the j -th eigenvalue associated to mass j has a closed form solution⁷⁷

$$\omega_j^2 = 4s \sin^2\left(\frac{j\pi}{2(n+1)}\right), \quad (83)$$

where s is the spring coefficient and n the number of masses. The maximal eigenvalue is thus given by

$$\omega_{\max}^2 = 4s \sin^2\left(\frac{n\pi}{2(n+1)}\right) \approx 4s, \quad (84)$$

and the radial frequency by

$$\omega_{\max} \approx 2\sqrt{s}. \quad (85)$$

The minimal eigenvalue corresponds to

$$\omega_{\min}^2 = 4s \sin^2\left(\frac{\pi}{2(n+1)}\right) \approx 4s\left(\frac{\pi}{2(n+1)}\right)^2, \quad (86)$$

and the radial frequency is thus given by

$$\omega_{\min} \approx \frac{\pi}{n+1}\sqrt{s}. \quad (87)$$

The range of spring coefficients can be determined by setting $\omega_{\min}$ (resp. $\omega_{\max}$) to the desired minimum (resp. maximum) radial frequency and solving for s .

For each data sample, corresponding to the displacement trajectory $X \in \mathbb{R}^{n \times T}$, we randomly generated an initial condition consisting of an initial displacement $x_{i,0}$ sampled from a standard normal distribution $\mathcal{N}(0,1)$ and zero-valued initial velocity, for each mass $i \in [1, n]$. We then simulated the temporal evolution of this system, such that the k -th column of X was given by displacements of the masses at time $k\Delta t$ with simulation time step $\Delta t = 2.5$ ms for 500 ms of simulation during training, totaling 200 time-steps. The vector $x[k]$ in

the main text was then given by the k -th column of X . The velocities were intentionally held out from the training data to increase the task difficulty and enforce the utilization of internal states by the neural network. For Fig. 4e, the model with 958 parameters corresponds to an adLIF network of 25 neurons. We doubled the number of neurons until 3200, neuron counts of other models (LIF and LSTM) were scaled such that these models match the number of trainable parameters in the corresponding adLIF network.

Details for the audio compression task

For the audio compression task from Section High-fidelity neuro-morphic audio compression, we used the ‘train-clean-100’ dataset from the LibriTTS corpus^{78,79}, comprising 53.78 hours of raw recorded speech data from audiobooks, split into 33, 200 samples of varying length in the interval of [0.16, 32] seconds. In this dataset, each sample is encoded with 16 bit Pulse Code-Modulation (PCM) sampled at 24 kHz. We first rescaled each sample x individually by dividing it by its peak amplitude $|\max(x)|$, ensuring that the rescaled sample had a maximum absolute amplitude of 1. This rescaling did not change the zero-amplitude level of the data. Next, we segmented each rescaled sample into non-overlapping blocks of 2560 time steps (equivalent to 106 ms) and treated each of these blocks as individual, independent sample $x_w \in \mathbb{R}^{2560}$ for training. Since the task was the reconstruction of the original signal, akin to an autoencoder, the input time series x_w and the target time series $y_w^* \in \mathbb{R}^{2560}$ were equivalent. We use the subscript w to denote that both x_w and y_w^* represent amplitudes in the waveform domain.

To train spiking neural networks for waveform reconstruction, we considered a spectral and a temporal loss. For the temporal loss, our objective was to maximize the likelihood of the target amplitude $y_w^*[k+1]$ at the $k+1$ -th time step, conditioned on the sequence of preceding amplitude values $(x_w[1], \dots, x_w[k])$. This optimization problem can be formalized to

$$\arg \max_{\theta} P(y_w^*; \theta) = \prod_{k=1}^{T-1} p(y_w^*[k+1] | x_w[1], \dots, x_w[k]; \theta), \quad (88)$$

where θ are the model parameters. In line with recent studies (e.g. Wavenet⁸⁰ and sampleRNN⁸¹), we quantize y_w^* into 256 discrete levels, treating p as a categorical distribution. The output layer of all our simulated networks consisted of 256 leaky-integrator (LI) neurons representing the logits associated with each of these discrete levels. The target class associated with each time step was defined by the quantization scheme:

$$y_{q[k]}^* = \left\lfloor \frac{f_A(y_w^*[k+1])}{\Delta_q} \right\rfloor, \quad (89)$$

where $\lfloor x \rfloor$ denotes the rounding operator, $\Delta_q = \frac{2}{2^8-1}$ is the discretization interval, and subscript q indicates the quantized, categorical representation. The transformation function $f_A(y_w[k])$ represents the A-law non-linearity⁸², defined as:

$$f_A(x) = \text{sign}(x) \begin{cases} \frac{A|x|}{1 + \ln(A)} & \text{if } |x| < \frac{1}{A}, \\ \frac{1 + \ln(A|x|)}{1 + \ln(A)} & \text{if } |x| \geq \frac{1}{A}, \end{cases} \quad (90)$$

where $A = 86.7$ is a hyperparameter, and $|x|$ denotes the absolute value of x . This type of invertible log-space mapping was previously used for audio generation in⁸⁰ and is a common technique used in 8 bit telephony to improve noise robustness⁸², as it is less sensitive to low amplitude noise while maintaining high precision for significant amplitude magnitudes. The original audio data is encoded with 16 bit, hence for perfect reconstruction with a categorical distribution this would require 65536 classes. By using a companding algorithm like the A-law, we limit the effects of reducing our precision to 8 bit. The

temporal loss is finally defined as

$$\mathcal{L}_{\text{temp}}(\mathbf{y}_q, \mathbf{y}_q^*) = \text{CrossEntropy}(\mathbf{y}_q, \mathbf{y}_q^*) \quad (91)$$

where $\mathbf{y}_q[k] = \text{softmax}(\frac{\mathbf{o}[k]}{\tau})$ of model output $\mathbf{o}[k] \in \mathbb{R}^{256}$ at time k and $\tau \geq 1$ is a temperature hyperparameter, which we initialized at $\tau = 10$ and reduced by a factor of 0.95 every 2000 training batches, until a minimum value of 1.

From this network output $\mathbf{y}_q[k]$ in the quantized domain we obtained the waveform-amplitude $y_w[k]$ by computing a convex sum over the quantization levels via

$$y_w[k] = f_A^{-1} \left(\sum_{i=1}^{256} (i-1) \cdot y_q^i[k] \cdot \Delta_q - 1 \right). \quad (92)$$

Here, f_A^{-1} denotes the inverse of the A-law operation (Eq. (90)) and superscript i denotes the i -th entry of the quantized vector. While temporal objectives, such as the one described above, have been used successfully in important studies^{80,81}, modern approaches to audio generation and compression generally rely on spectral objectives^{47,83}. In this work we found that combining spectral and temporal objectives gives the best results. From the reconstructed waveform y_w and the target waveform y_w^* , we computed a spectral loss based on a multi-resolution Mel-spectral short-term Fourier transform (STFT) loss⁸⁴ defined as

$$\mathcal{L}_w = \frac{1}{6} \sum_{k \in \{2^0, 2^1, \dots, 2^H\}} (\mathcal{L}_{\text{lin}}^k + \mathcal{L}_{\text{log}}^k), \quad (93)$$

where

$$\mathcal{L}_{\text{lin}}^k(\mathbf{y}_v, \mathbf{y}_w^*) = \frac{1}{TN} \|\text{STFT}_{\text{Mel}}^k(\mathbf{y}_v) - \text{STFT}_{\text{Mel}}^k(\mathbf{y}_w^*)\|_1 \quad (94)$$

$$\mathcal{L}_{\text{log}}^k(\mathbf{y}_v, \mathbf{y}_w^*) = \frac{1}{TN} \|\ln(|\text{STFT}_{\text{Mel}}^k(\mathbf{y}_v)|) - \ln(|\text{STFT}_{\text{Mel}}^k(\mathbf{y}_w^*)|)\|_1. \quad (95)$$

Here, $\text{STFT}_{\text{Mel}}^k$ is the unnormalized short-term Fourier transform operator over a k -length Hann window using the HTK-variant of the Mel frequency filters. $\|\cdot\|_1$ denotes the $L1$ norm, T corresponds to the number of STFT frames, and N the number of Mel filters ($N = 128$). We consider a hop length of $k/4$ and the number of frequencies for the STFT was set to 2048. The overall loss was defined as

$$\mathcal{L}_{\text{total}}(\mathbf{y}_q, \mathbf{y}_w^*) = \mathcal{L}_w(\mathbf{y}_w, \mathbf{y}_w^*) + \mathcal{L}_{\text{temp}}(\mathbf{y}_q, \mathbf{y}_q^*). \quad (96)$$

For every sample, during training, we ignored the model output during the first 50 time steps (i.e. 2 ms) as burn-in time for the model.

For all simulations (LIF, SE-adLIF, and EF-adLIF), the networks consisted of two fully-connected encoder layers, where only the second layer included recurrent connections. From the second layer, the first 16 neurons were connected to the decoder network, enforcing the information bottleneck. The decoder network was also a two-layer network, with recurrent connections in both layers, followed by an additional non-recurrent output layer of 256 leaky integrator (LI) neurons.

The general hyperparameters for all models can be found in Table 4. The difference with other tasks is that the initial learning rate of $5 \cdot 10^{-4}$ was reduced by a factor of 0.1 if the SI-SNR on the validation set did not improve after a training epoch. However, the learning rate was not reduced further than 10^{-7} . For the SE-adLIF model in particular, in order to increase intrinsic frequency range, we increase the range of

$\hat{a}$ from $[0, 1]$ to $[0, 5]$ we find this more effective than directly increasing the scaling coefficient q .

For each time step k , the network output $\mathbf{y}[k] \in \mathbb{R}^{256}$ was constructed by passing the output of the LI layer through a softmax.

In this task, we employed a small modification to the reset mechanism of the LIF and adLIF neuron models: Instead of resetting the membrane potential to 0 after a spike, we reset it to a learnable reset potential u_{reset} . Each neuron thereby had a separate value for u_{reset} , clipped to the interval $[-\vartheta, \vartheta]$. In addition, we trained the spike threshold $\vartheta \in \mathbb{R}^+$ of each individual neuron, instead of treating it as fixed hyperparameter, as done in all our other tasks. We found that these modifications substantially improved the performance of the SNN networks. Another modification we found beneficial was to delay the target time series by a delay of 20 time steps (≈ 0.8 ms), such that the model had more ‘time’ to encode the waveform into spikes. In other words, at time step k , the prediction target for the model was $\mathbf{y}_q^*[k-20]$. During training, we applied spike regularization to each layer l of the network according to:

$$\mathcal{L}_{\text{reg}}(r_l^n) = \begin{cases} g_l^+(r_l^n - t_l^+)^2 & \text{if } r_l^n > t_l^+, \\ g_l^-(r_l^n - t_l^-)^2 & \text{if } r_l^n < t_l^-, \\ 0 & \text{otherwise,} \end{cases} \quad (97)$$

where r_l^n is the average number of spikes of neuron n in layer l over time, and $t_l^+, t_l^-, g_l^+, g_l^-$ are layer-specific hyperparameters. t_l^+ was set to 0.1 (adLIF), 0.5 (LIF) for the first layer of the encoder and decoder, 0.012 for the second layer of the encoder and 0.6 for the second layer of the decoder. t_l^- was set to 0.05 except for the second encoder layer, which was set to 0.005. g_l^+ was set to 10 except for the second encoder layer where it set to 100. g_l^- was set to 10. For the second encoder layer, the spike regularization was only applied to the first 16 neurons, which constitute the bottleneck and are the only neurons connected to the decoder. $\mathcal{L}_{\text{reg}}$ was added to the total loss from Eq. (96).

Details for simulations in Fig. 5

The neuron parameters for the experiments in Fig. 5 were $\tau_u = 100$ ms for LIF and $\tau_u = 100$ ms, $\tau_w = 300$ ms, $a = 300$ for adLIF, resulting in an adLIF neuron with an intrinsic oscillation frequency $f \approx 16$ Hz. We used the first derivative of a Gaussian function as wavelet, which was scaled such that the central oscillation frequency was ≈ 17 Hz. The offsets for the wavelet in Fig. 5e were -32 ms and -48 ms as $-\frac{1}{2}P$ and $-\frac{3}{4}P$ respectively. The loss was evaluated at time $T = 330$ ms.

Details for the burst sequence detection (BSD) task

The BSD task from Fig. 6a, b is a 20-class classification task consisting of 8000 samples. Each binary-valued sample $X \in [0, 1]^{T \times N}$ in this task consists of spike trains of $N = 10$ input neurons and a time duration of $T = 200$ ms in discrete 1 ms time steps. The objective in this task is to classify any sample X based on the appearance of spike bursts of specific neurons at specific timings. The timing and neurons for these class-descriptive bursts were pre-assigned upfront and kept fixed for the generation of the entire dataset. To generate this dataset, we employed a two-step process where we first randomly assigned class-descriptive burst timings for each class, then stochastically sampled data samples based on these pre-assigned timings. In detail, this procedure was as follows: For the pre-assignment of burst timings, we sampled a random subset S_c of 3 input neurons for each class c and sampled a random time point t_c^n uniformly over $[20, 170]$ for each neuron $n \in S_c$ for class c . These time points served as the class-descriptive burst timings.

After generating the class-specific burst-timings for all classes, the data samples were generated. To generate a sample of a given class c , we first determined the burst timing for each neuron $n \in [1, M]$ as follows. If neuron n is one of the 3 neurons inside the pre-defined set S_c of class c , then its burst-timing t^n is determined by the pre-assigned timing t_c^n . Otherwise, we randomly selected a burst timing t^n uniformly over $[20, 170]$ for neuron n . In the example shown in Fig. 6a, a sample of class 2 is defined by a spike burst of neuron 4 at 159 ms, a burst of neuron 5 at 143 ms, and a burst of neuron 10 at 117 ms. Hence, the set S_2 of class-descriptive neurons for class 2 is $\{4, 5, 10\}$ with burst timings $t_2^4 = 159$, $t_2^5 = 143$ and $t_2^{10} = 117$. Only if all of the neurons assigned to S_c emit a burst at their corresponding timings t_c^n , the sample should be classified as class c . All other input neurons show distraction bursts at random timings.

In an input sample $X \in [0, 1]^{T \times N}$, $x_{t,n} = 1$ indicates a spike of input neuron n at time t ($x_{t,n} = 0$ if there is no spike). We generated the spike trains X for input neurons for a given class c as follows. Each $x_{t,n}$ was drawn from a Bernoulli distribution with $p(x_{t,n} = 1|c)$ that was obtained as follows. Bursts were modeled as brief, smooth increases in spike probability at the corresponding times t^n . We achieved this by a Gaussian function $f(t, t^n) = \exp\left(\frac{-(t-t^n)^2}{4}\right)$, yielding high spike probability for time steps close to t^n , and lower spike probability further away. From this function, we computed the final spike probability $p(x_{t,n} = 1|c) = \frac{f(t, t^n)}{\max_{k \in [0, T]} f(k, t^n)} \cdot 0.75 + 0.05$, interpolating between the minimum and maximum spike probabilities of 0.05 (50 Hz) and 0.8 (800 Hz) respectively. This way we ensured that the spike probability went up to 0.8 during a burst and was approximately 0.05 otherwise to mimic background noise.

This procedure was repeated for each individual data sample X to obtain the dataset. We randomly chose a class for each new sample, such that the class cardinalities in the dataset were roughly but not exactly balanced. The whole data set consisted of 8000 samples. We held out 10% (800 samples) of this dataset for validation and 20% (1600 samples) for testing. For the experiment with different numbers of classes (Fig. 6b), we only increased the number of classes, but did not increase the number of samples in the dataset. This resulted in increased difficulty for larger numbers of classes, since less samples per class were present in the training data.

Optimization-based feature visualization

The visualization of important input features for the trained network shown in Fig. 6 was obtained using optimization-based feature visualization as defined in⁵².

We applied the same sampling method to four different networks, one adLIF and one LIF network trained on BSD and one adLIF and one LIF trained on SHD. For both cases we used the same algorithm, but with different parameters: $n_{\text{iter}} = 400$ (BSD), $n_{\text{iter}} = 200$ (SHD), $\eta = 0.1$, $\nu = 0.891$, $\gamma = 0.1$, and $\sigma_{\text{init}} = 5$.

For both tasks, we selected network instances that achieved high accuracy. The sampling procedure was performed as follows. First, we initialized a random input example $X^0 \in \mathbb{R}^{T \times C}$ with C as the input dimension and T the input length (number of time steps), both of the same dimension as the data the network was trained on. Here, each $x_{t,c}^0$ was drawn from the uniform distribution $\mathcal{U}(0, 1)$. At each iteration k , the sample was passed through the network and the loss was calculated akin to the loss function used during training on the specific dataset (SHD or BSD) to obtain the gradient $G_k = \nabla_X L(X, c)|_{X^k}$, with respect to a pre-defined target class c . This gradient was normalized to obtain $\Delta X^k = \eta \frac{G_k}{\zeta_k}$ with normalization factor $\zeta_k = \max\{|G_k|_{\max}, \epsilon\}$ using an $\epsilon = 10^{-6}$ for numerical stability, and a step size η . $|\cdot|_{\max}$ denotes the maximum norm returning the highest absolute value. After the gradient update, we clipped the sample X^{k+1} to the positive range, as the

data from both datasets, SHD and BSD, is all-positive. We then applied Gaussian smoothing on the data sample X^{k+1} . In the BSD case we applied a 1D smoothing along the time axis using a Gaussian kernel $g(s) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{s^2}{2\sigma^2}\right)$ via $X^{k+1} \leftarrow \nu X^{k+1} + \gamma(g * X^{k+1})$. Decay $\nu \in [0, 1]$ and smoothing coefficient $\gamma \in [0, 1]$ are hyperparameters, $*$ denotes a convolution operation. The σ parameter of the Gaussian kernel was linearly decayed from σ_{init} to 0 over the n_{iter} iterations to decrease the effective regularization applied by the kernel smoothing. For the SHD case we applied a 2D smoothing to the data sample over both the time dimension and the spatial dimension. For the BSD case, we normalized the sample X^{k+1} with $X^{k+1} \leftarrow X^{k+1} \frac{\mu_{\text{all}}}{\mu_{k+1}}$, where scalar μ_{all} is the mean spike rate over all data samples and input neurons in the data, and μ_{k+1} the mean of sample X^{k+1} over both the time and the input neuron axes. For the SHD setup, we ignored the loss for the first 100 sequence time steps compare with gray shaded area in Fig. 6e, whereas for the BSD case we ignored the loss for the first 80% of time steps, which was also done during training of the network on the dataset (see Section Training and Hyperparameter Search Details for all Tasks).

Data availability

The SHD and SSC datasets are publicly available at <https://zenkelab.org/resources/spiking-heidelberg-datasets-shd/> under CC BY 4.0 license. The QTDB ECG dataset is publicly available at <https://physionet.org/content/qtdb/1.0.0/> under ODC BY 1.0 license. However, we used the preprocessed files from²⁵. LibriTTS corpus is publicly available at <https://www.openslr.org/60/> under CC BY 4.0 license.

Code availability

The code is available under the CC BY-SA 4.0 license at the repository <https://github.com/IGITUGraz/SE-adlif> and as a permanent archive at⁸⁵.

References

- Maass, W. Networks of spiking neurons: The third generation of neural network models. *Neural Netw.* **10**, 1659–1671 (1997).
- Schemmel, J. et al. A wafer-scale neuromorphic hardware system for large-scale neural modeling. In *2010 IEEE International Symposium on Circuits and Systems (ISCAS)*, 1947–1950 (2010).
- Furber, S. B., Galluppi, F., Temple, S. & Plana, L. A. The spinnaker project. *Proc. IEEE* **102**, 652–665 (2014).
- Merolla, P. A. et al. A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science* **345**, 668–673 (2014).
- Davies, M. et al. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro* **38**, 82–99 (2018).
- Davies, M. et al. Advancing neuromorphic computing with loihi: A survey of results and outlook. *Proc. IEEE* **109**, 911–934 (2021).
- Lee, J. H., Delbruck, T. & Pfeiffer, M. Training deep spiking neural networks using backpropagation. *Front. Neurosci.* **10**, 508 (2016).
- Shrestha, S. B. & Orchard, G. Slayer: Spike layer error reassignment in time. In Bengio, S. et al. (eds.) *Advances in Neural Information Processing Systems*, vol. 31 (Curran Associates, Inc., 2018).
- Gerstner, W., Kistler, W. M., Naud, R. & Paninski, L. *Neuronal dynamics: From single neurons to networks and models of cognition* (Cambridge University Press, 2014).
- Hodgkin, A. L. The local electric changes associated with repetitive action in a non-medullated axon. *J. Physiol.* **107**, 165–181 (1948).
- Izhikevich, E. M. Neural excitability, spiking and bursting. *Int. J. Bifurc. Chaos* **10**, 1171–1266 (2000).
- Izhikevich, E. M. Resonate-and-fire neurons. *Neural Netw.* **14**, 883–894 (2001).
- Bellec, G., Salaj, D., Subramoney, A., Legenstein, R. & Maass, W. Long short-term memory and learning-to-learn in networks of spiking neurons. In Bengio, S. et al. (eds.) *Advances in Neural*

- Information Processing Systems, vol. 31, 787–797 (Curran Associates, Inc., 2018).
14. Salaj, D. et al. Spike frequency adaptation supports network computations on temporally dispersed information. *Elife* **10**, e65459 (2021).
 15. Ganguly, C. et al. Spike frequency adaptation: Bridging neural models and neuromorphic applications. *Commun. Eng.* **3**, 1–13 (2024).
 16. Bittar, A. & Garner, P. N. A surrogate gradient spiking baseline for speech command recognition. *Front. Neurosci.* **16**, 865897 (2022).
 17. Higuchi, S., Kairat, S., Bohte, S. M. & Otte, S. Balanced resonate-and-fire neurons (2024). 2402.14603.
 18. Guo, Y. et al. Membrane potential batch normalization for spiking neural networks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 19420–19430 (2023).
 19. Jiang, H., Zoonekynd, V., Masi, G. D., Gu, B. & Xiong, H. TAB: Temporal Accumulated Batch Normalization in Spiking Neural Networks. In *The Twelfth International Conference on Learning Representations* (2023).
 20. Zheng, H., Wu, Y., Deng, L., Hu, Y. & Li, G. Going deeper with directly-trained larger spiking neural networks. *Proc. AAAI Conf. Artif. Intell.* **35**, 11062–11070 (2021).
 21. Vicente-Sola, A., Manna, D. L., Kirkland, P., Di Caterina, G. & Bihl, T. Spiking Neural Networks for event-based action recognition: A new task to understand their advantage (2023). 2209.14915.
 22. Rath, N. et al. Exploring neuromorphic computing based on spiking neural networks: Algorithms to hardware. *ACM Comput. Surv.* **55**, 1–49 (2023).
 23. Brunel, N., Hakim, V. & Richardson, M. J. E. Firing-rate resonance in a generalized integrate-and-fire neuron with subthreshold resonance. *Phys. Rev. E* **67**, 051916 (2003).
 24. Deckers, L., Van Damme, L., Van Leekwijck, W., Tsang, I. J. & Latré, S. Co-learning synaptic delays, weights and adaptation in spiking neural networks. *Front. Neurosci.* **18**, 1360300 (2024).
 25. Yin, B., Corradi, F. & Bohté, S. M. Accurate and efficient time-domain classification with adaptive spiking recurrent neural networks. *Nat. Mach. Intell.* **3**, 905–913 (2021).
 26. Wang, Y. & Howard, N. The spike frequency modulation (sfm) theory for neuroinformatics and cognitive cybernetics. In *2019 IEEE International Conference on Systems, Man and Cybernetics (SMC)*, 2220–2224 (2019).
 27. Abadi, M. et al. TensorFlow: Large-scale machine learning on heterogeneous systems (2015). Software available from tensorflow.org.
 28. Ansel, J. et al. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 929–947 (ACM, La Jolla CA USA, 2024).
 29. Orchard, G. et al. Efficient Neuromorphic Signal Processing with Loihi 2 (2021). 2111.03746.
 30. Gu, A. et al. Combining Recurrent, Convolutional, and Continuous-time Models with Linear State-Space Layers (2021). 2110.13985.
 31. Gu, A. & Dao, T. Mamba: Linear-Time Sequence Modeling with Selective State Spaces (2023). 2312.00752.
 32. Hairer, E., Lubich, C. & Wanner, G. *Geometric numerical integration*, vol. 31 of *Springer Series in Computational Mathematics* (Springer-Verlag, Berlin, 2006), second edn. Structure-preserving algorithms for ordinary differential equations.
 33. Effenberger, F., Carvalho, P., Dubinin, I. & Singer, W. The functional role of oscillatory dynamics in neocortical circuits: A computational perspective. *Proc. Natl. Acad. Sci. USA*. **122**, e2412830122 (2025).
 34. Brette, R. & Gerstner, W. Adaptive exponential integrate-and-fire model as an effective description of neuronal activity. *J. Neurophysiol.* **94**, 3637–3642 (2005).
 35. Cramer, B., Stradmann, Y., Schemmel, J. & Zenke, F. The Heidelberg spiking data sets for the systematic evaluation of spiking neural networks. *IEEE Trans. Neural Netw. Learn. Syst.* **33**, 2744–2757 (2022).
 36. Neftci, E. O., Mostafa, H. & Zenke, F. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Process. Mag.* **36**, 51–63 (2019).
 37. Perez-Nieves, N., Leung, V. C. H., Dragotti, P. L. & Goodman, D. F. M. Neural heterogeneity promotes robust learning. *Nat. Commun.* **12**, 5791 (2021).
 38. Hammouamri, I., Khalfouli-Hassani, I. & Masquelier, T. Learning delays in spiking neural networks using dilated convolutions with learnable spacings (2023). 2306.17670.
 39. Roth, F. et al. Spike-based sensing and communication for highly energy-efficient sensor edge nodes. In *2022 2nd IEEE International Symposium on Joint Communications & Sensing (JC&S)*, 1–6 (2022).
 40. Wu, D., Chen, J., Rajendran, B., Poor, H. V. & Simeone, O. Neuromorphic wireless split computing with multi-level spikes (2024). 2411.04728.
 41. Ke, Y. et al. Neuromorphic Wireless Device-Edge Co-Inference via the Directed Information Bottleneck. In *2024 International Conference on Neuromorphic Systems (ICONS)*, 16–23 (IEEE Computer Society, Los Alamitos, CA, USA, 2024). <https://doi.org/10.1109/ICONS62911.2024.00011>.
 42. Chen, J., Skachkovsky, N. & Simeone, O. Neuromorphic wireless cognition: Event-driven semantic communications for remote inference. *IEEE Trans. Cogn. Commun. Netw.* **9**, 252–265 (2023).
 43. Savazzi, P., Vizziello, A. & Dell’Acqua, F. Mutual information analysis of neuromorphic coding for distributed wireless spiking neural networks (2024). 2405.18019.
 44. Fernandes, J. R. & Wentzloff, D. Recent advances in IR-UWB transceivers: An overview. In *Proceedings of 2010 IEEE International Symposium on Circuits and Systems*, 3284–3287 (IEEE, Paris, France, 2010).
 45. Valin, J.-M., Vos, K. & Terriberry, T. Definition of the opus audio codec. Tech. Rep. (2012).
 46. Dietz, M. et al. Overview of the evs codec architecture. In *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 5698–5702 (IEEE, 2015).
 47. Défossez, A., Copet, J., Synnaeve, G. & Adi, Y. High fidelity neural audio compression. *arXiv preprint arXiv:2210.13438* (2022).
 48. Luo, Y. & Mesgarani, N. Tasnet: time-domain audio separation network for real-time, single-channel speech separation. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 696–700 (IEEE, 2018).
 49. Hines, A., Skoglund, J., Kokaram, A. C. & Harte, N. Visqol: an objective speech quality model. *EURASIP Journal on Audio, Speech, and Music Processing* **2015**, 1–18 (2015).
 50. Harvey, C. D., Coen, P. & Tank, D. W. Choice-specific sequences in parietal cortex during a virtual-navigation decision task. *Nature* **484**, 62–68 (2012).
 51. Lisman, J. E. Bursts as a unit of neural information: making unreliable synapses reliable. *Trends Neurosci.* **20**, 38–43 (1997).
 52. Erhan, D., Bengio, Y., Courville, A. & Vincent, P. Visualizing higher-layer features of a deep network. *Univ. Montr.* **1341**, 1 (2009).
 53. Ioffe, S. & Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *CoRRabs/1502.03167* (2015). 1502.03167.

54. Frenkel, C. & Indiveri, G. Reckon: A 28nm sub-mm² task-agnostic spiking recurrent neural network processor enabling on-chip learning over second-long timescales. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, 1–3 (IEEE, 2022).
55. Kilner, J. M., Baker, S. N., Salenius, S., Hari, R. & Lemon, R. N. Human cortical muscle coherence is directly related to specific motor parameters. *J. Neurosci.* **20**, 8838–8845 (2000).
56. Sanes, J. N. & Donoghue, J. P. Oscillations in local field potentials of the primate motor cortex during voluntary movement. *Proc. Natl Acad. Sci. USA* **90**, 4470–4474 (1993).
57. Raissi, M., Perdikaris, P. & Karniadakis, G. E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* **378**, 686–707 (2019).
58. Rusch, T. K. & Mishra, S. Coupled oscillatory recurrent neural network (cornn): An accurate and (gradient) stable architecture for learning long time dependencies. In *International Conference on Learning Representations* (2021).
59. Dubinin, I. & Effenberger, F. Fading memory as inductive bias in residual recurrent networks. *Neural Netw.* **173**, 106179 (2024).
60. Glorot, X. & Bengio, Y. Understanding the difficulty of training deep feedforward neural networks. In *AISTATS* (2010).
61. Voelker, A., Kajić, I. & Eliasmith, C. Legendre Memory Units: Continuous-Time Representation in Recurrent Neural Networks. In *Advances in Neural Information Processing Systems*, vol. 32 (Curran Associates, Inc., 2019).
62. Gupta, A., Gu, A. & Berant, J. Diagonal State Spaces are as Effective as Structured State Spaces (2022). 2203.14343.
63. Smith, J. T. H., Warrington, A. & Linderman, S. W. Simplified State Space Layers for Sequence Modeling (2023). 2208.04933.
64. Orvieto, A. et al. Resurrecting Recurrent Neural Networks for Long Sequences. In *Proceedings of the 40th International Conference on Machine Learning*, 26670–26698 (PMLR, 2023).
65. Elman, J. L. Finding structure in time. *Cogn. Sci.* **14**, 179–211 (1990).
66. Maass, W. & Bishop, C. M. *Pulsed neural networks* (MIT press, 2001).
67. Bal, M. & Sengupta, A. Rethinking spiking neural networks as state space models (2024). 2406.02923.
68. Schöne, M. et al. Scalable event-by-event processing of neuro-morphic sensory signals with deep state-space models (2024). 2404.18508.
69. Zheng, H. et al. Temporal dendritic heterogeneity incorporated with spiking neural networks for learning multi-timescale dynamics. *Nat. Commun.* **15**, 277 (2024).
70. Ferrand, R., Baronig, M., Limbacher, T. & Legenstein, R. Context-dependent computations in spiking neural networks with apical modulation. In *International Conference on Artificial Neural Networks*, 381–392 (Springer, 2023).
71. Hespanha, J. P. *Linear Systems Theory: Second Edition* (Princeton University Press, 2018), ned - new edition, 2 edn.
72. Laguna, P., Mark, R., Goldberg, A. & Moody, G. A database for evaluation of algorithms for measurement of qt and other wave-form intervals in the eeg. In *Computers in Cardiology 1997*, 673–676 (1997).
73. Kingma, D. P. & Ba, J. Adam: A method for stochastic optimization (2017). 1412.6980.
74. Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A. & Talwalkar, A. Hyperband: A novel bandit-based approach to hyperparameter optimization. *J. Mach. Learn. Res.* **18**, 1–52 (2018).
75. Jozefowicz, R., Zaremba, W. & Sutskever, I. An empirical exploration of recurrent network architectures. In *International conference on machine learning*, 2342–2350 (PMLR, 2015).
76. Saxe, A., McClelland, J. & Ganguli, S. Exact solutions to the non-linear dynamics of learning in deep linear neural networks (International Conference on Learning Representations 2014, 2014).
77. Kulkarni, D., Schmidt, D. & Tsui, S.-K. Eigenvalues of tridiagonal pseudo-toeplitz matrices. *Linear Algebra Appl.* **297**, 63–80 (1999).
78. Zen, H. et al. Libritts: A corpus derived from librispeech for text-to-speech. *arXiv preprint arXiv:1904.02882* (2019).
79. Panayotov, V., Chen, G., Povey, D. & Khudanpur, S. Librispeech: An ASR corpus based on public domain audio books. In *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 5206–5210 (2015).
80. Van Den Oord, A. et al. Wavenet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499* (2016).
81. Mehri, S. et al. SampleRNN: An Unconditional End-to-End Neural Audio Generation Model (2017). 1612.07837.
82. ITU. Uit-t, g.711. https://web.archive.org/web/2025000000000000*/https://www.itu.int/rec/T-REC-G.711/. Accessed: 24-01-2025.
83. Zeghidour, N., Luebs, A., Omran, A., Skoglund, J. & Tagliasacchi, M. SoundStream: An End-to-End Neural Audio Codec (2021). 2107.03312.
84. Gritsenko, A., Salimans, T., van den Berg, R., Snoek, J. & Kalchbrenner, N. A spectral energy distance for parallel speech synthesis. *Adv. Neural Inf. Process. Syst.* **33**, 13062–13072 (2020).
85. Baronig, M., Ferrand, R., Sabathiel, S. & Legenstein, R. Advancing spatio-temporal processing through adaptation in spiking neural networks (2025). <https://doi.org/10.5281/zenodo.15364925>.

Acknowledgements

This work has been supported by the “University SAL Labs” initiative of Silicon Austria Labs (SAL) and its Austrian partner universities for applied fundamental research for electronic based systems (M.B., R.F., S.S.), by the European Community’s Horizon 2020 FET-Open Program, grant number 899265, ADOPD (R.L., M.B., R.F.), and by NSF EFRI grant #2318152 (R.L.). This research was funded in whole or in part by the Austrian Science Fund (FWF) [10.55776/COE12] (R.L., M.B.). For the purpose of open access, the author has applied a CC BY public copyright licence to any Author Accepted Manuscript version arising from this submission. We thank Felix Effenberger and Sebastian Otte for valuable feedback on the manuscript.

Author contributions

M.B., R.F., and R.L. conceived the idea and proposed the research, M.B., R.F. and S.S. carried out experiments, M.B. and R.F. performed theoretical analyses, M.B., R.F., and R.L. wrote the paper.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s41467-025-60878-z>.

Correspondence and requests for materials should be addressed to Robert Legenstein.

Peer review information : *Nature Communications* thanks Lei Deng and the other, anonymous, reviewer(s) for their contribution to the peer review of this work. A peer review file is available.

Reprints and permissions information is available at <http://www.nature.com/reprints>

Publisher’s note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025